

PADDYGUARD AI

COMPLETE INTEGRATED DEVELOPMENT & DEPLOYMENT REPORT

Project ID: R26-SE-015 | October 2026

Team Members.

Component	Student name	IT number
C1 – Voice-based diagnosis	Weerasinghe K G J P	IT22273680
C2 – Leaf disease classification	Hewanayake H M L M	IT22168740
C3 – Pest detection	Kalhara E P K	IT22065858
C4 – Treatment advisory chatbot	Keshan B K	IT22303820

October 2026

1. Shared cloud deployment and frontend

1.1 Integrated deployment overview

The PaddyGuard AI application combines a React/Vite/TypeScript frontend, a central FastAPI API Gateway and four independently deployable research services. The Gateway routes browser requests to voice-NLP, rice leaf classification, pest detection and treatment advisory services. Supporting services include user management and PostgreSQL; component-specific integrations include Modal ASR, Redis, MongoDB Atlas, Cloudinary and RAG infrastructure as described in the original reports.

1.2 Hosting and service responsibilities

Layer	Hosting / technology	Purpose
Frontend	Vercel; React, Vite, TypeScript	Public browser interface
API Gateway	Railway; FastAPI	Routes public API requests
C1 voice-NLP	Railway; FastAPI; Modal Whisper ASR	Speech recognition and symptom diagnosis
C2 leaf disease	Railway; Flask/Gunicorn; PyTorch	Leaf disease inference and expert review
C3 pest detection	Independent hosted pest service	Pest image inference
C4 treatment advisory	Railway / independent team deployment; Node.js/Express	Retrieval-augmented guidance
User management	Railway; FastAPI; PostgreSQL	Accounts and authentication

1.3 Frontend deployment on Vercel

The frontend is a React/Vite/TypeScript single-page application. For cloud deployment, the frontend repository is connected to Vercel and configured to build the production bundle. Browser requests should target the public Railway Gateway instead of private internal Railway service URLs. The exact deployed Vercel domain and deployment status should be confirmed from the live dashboard before final submission.

Repository and build

Import the frontend GitHub repository into Vercel. Select the correct root directory. Use the project's Vite build command (typically npm run build) and output directory dist; confirm against package.json.

Environment configuration

Set VITE_API_URL to the public HTTPS Gateway base URL if that is the variable consumed by the frontend. If separate VITE_* API variables are used by individual pages, document and verify each actual variable against the frontend source. Never place secret tokens or private Railway hostnames in VITE_* variables.

Gateway integration

Allow the exact Vercel origin in Gateway CORS configuration. Verify /api/v1/voice/diagnose, /api/v1/image routes, pest detection, treatment/chat and login using the production browser application.

Validation

Test page refresh and direct routes, microphone permission, audio upload, image upload, pest image submission, chatbot replies, authentication, mobile layout and API failure handling. Record Vercel deployment status, public URL and browser/network evidence.

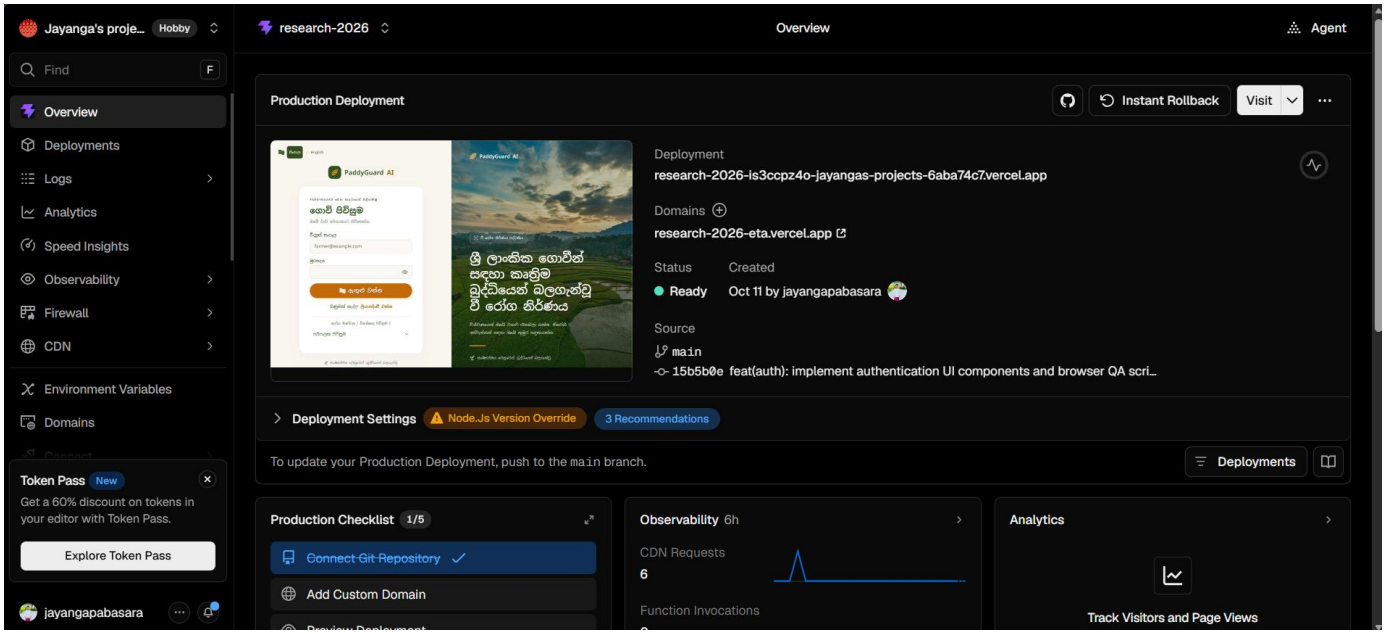
Operational security

Use HTTPS, environment-based configuration, secret management on server-side services, restrictive CORS, and error handling. Redeploy after changing Vercel environment variables.

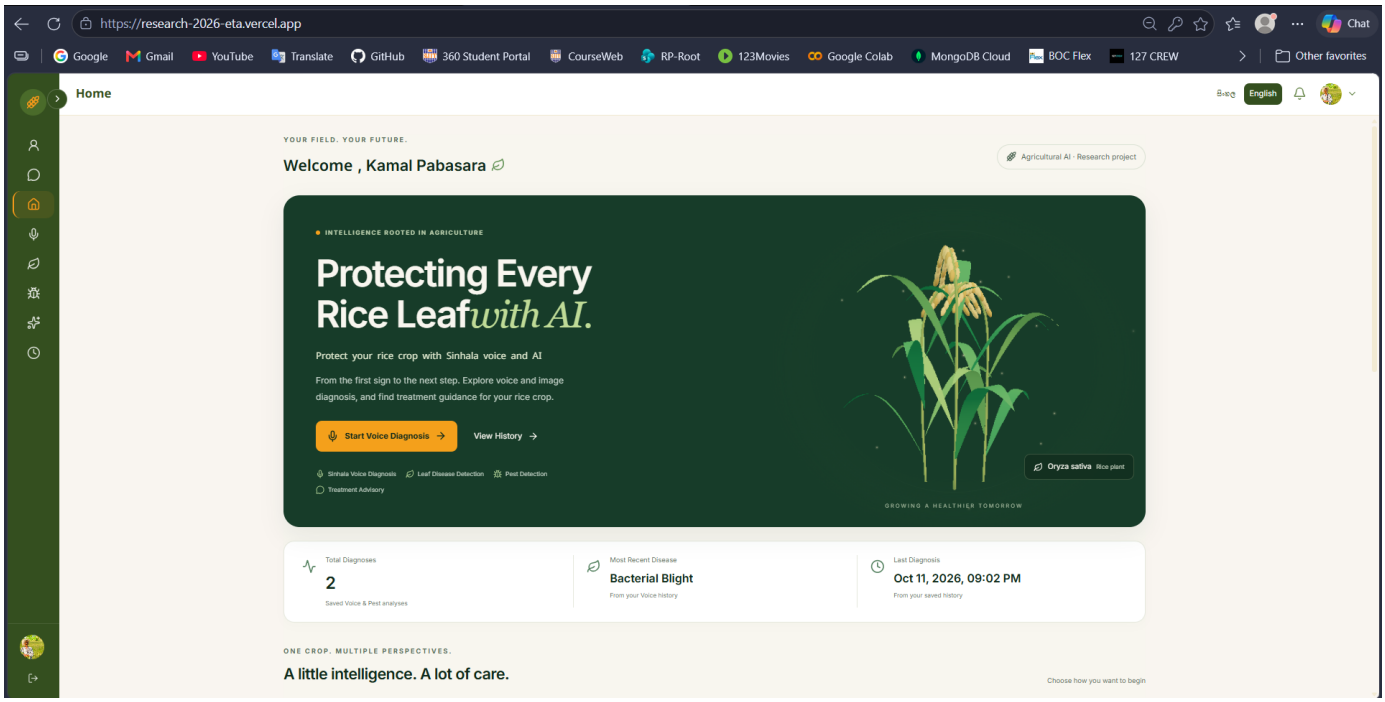
1.4 Frontend deployment evidence

Vercel project name: **research-2026**

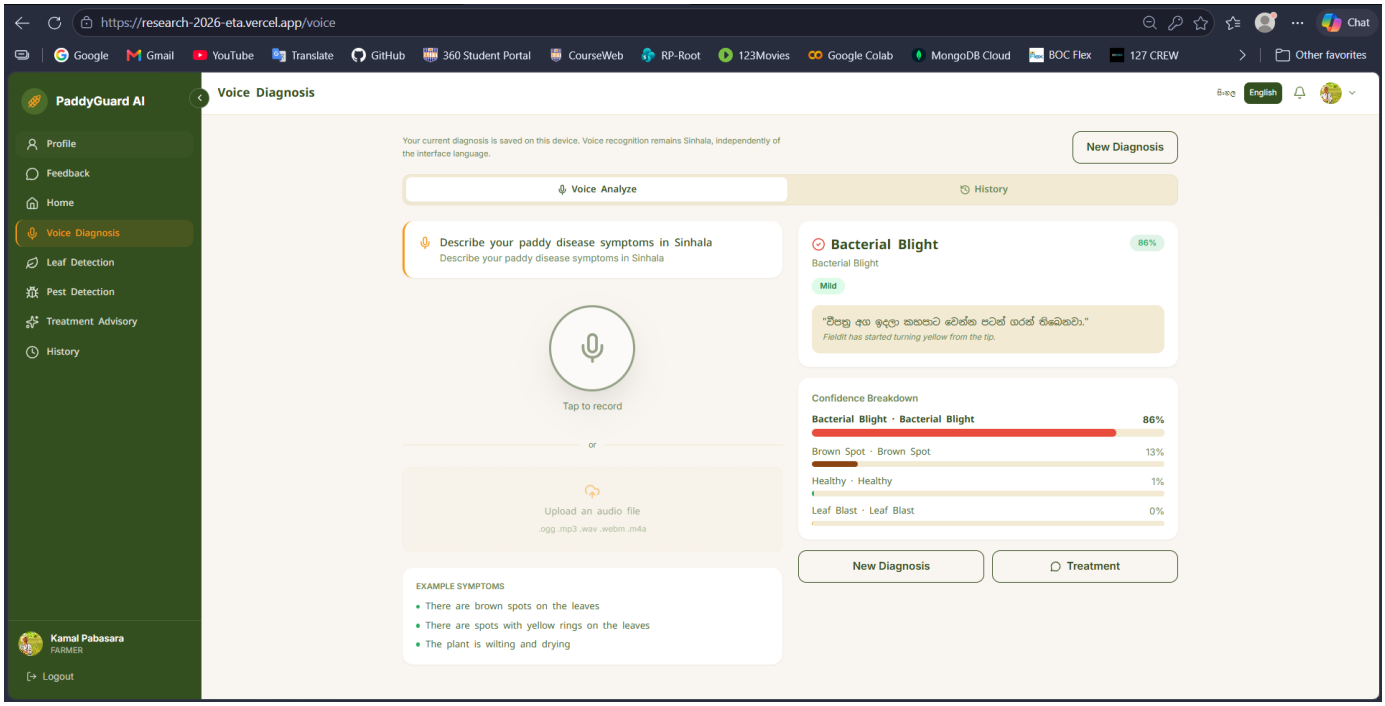
Production frontend URL: https://research-2026-eta.vercel.app/



Vercel deployment overview

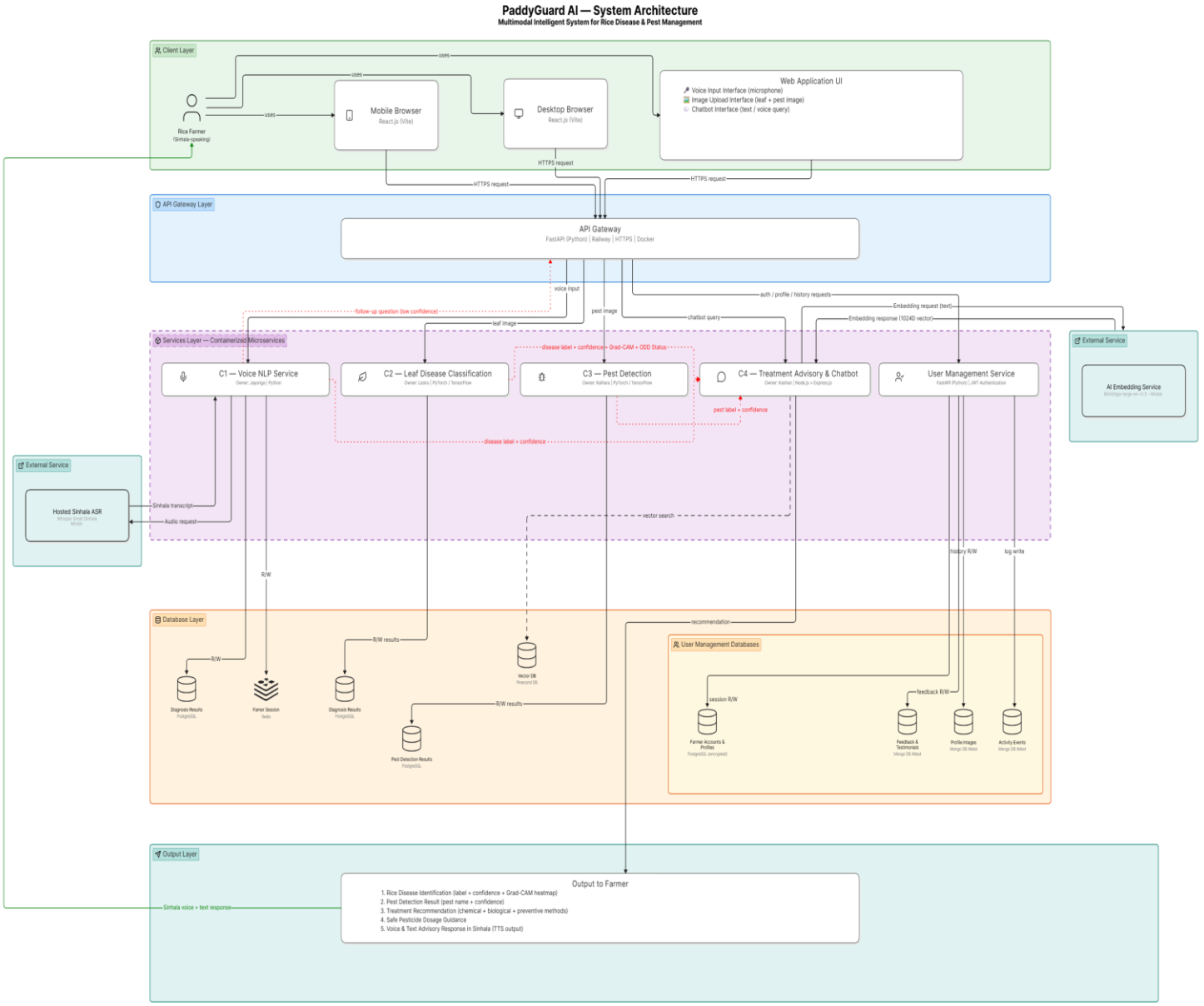


Deployed frontend homepage



End-to-end API workflow

1.5 Integrated deployment architecture figure



PaddyGuard AI integrated cloud deployment architecture

2. Component 1 — Voice-Based Rice Disease Diagnosis

PaddyGuard AI

A Multimodal AI-Based System for Rice Leaf Disease & Pest Detection, Treatment Recommendation and Knowledge Assistance

DEVELOPMENT REPORT

Component 1: Voice-Based Rice Disease Diagnosis using Speech Recognition and Natural Language Processing

Project ID	R26-SE-015
Student	Weerasinghe K G J P
Student ID	IT22273680
Degree	B.Sc. (Hons) Degree in Information Technology
Specialization	Software Engineering
Department	Department of Information Technology
Institution	Sri Lanka Institute of Information Technology (SLIIT), Sri Lanka
Supervisor	Dr. Nathali Silva
Co-supervisor	Ms. Poorna Panduwawala
Report Date	October 2026

Submitted as a development-progress report for the undergraduate research project.

Declaration

I declare that this development report documents the current state of my undergraduate research work on the Voice-Based Rice Disease Diagnosis component of PaddyGuard AI. Material derived from the approved project proposal and cited literature is acknowledged in the references. Development results described as completed are limited to work that has been implemented or functionally tested at the time of writing; proposal targets that still require formal experimental evaluation are identified as planned or pending.

Student: Weerasinghe K G J P
Student ID: IT22273680
Signature: _____
Date: _____

Supervisor acknowledgement (if required by the department):
Name: Dr. Nathali Silva
Signature: _____
Date: _____

Acknowledgement

I would like to express my sincere gratitude to my supervisor, Dr. Nathali Silva, and co-supervisor, Ms. Poorna Panduwawala, for their guidance and feedback throughout this research. I also acknowledge the contribution of the PaddyGuard AI research group, whose individual components form the integrated multimodal platform. Appreciation is extended to agricultural domain experts and the Department of Agriculture of Sri Lanka for the domain context that informs the disease categories and farmer-facing design of the proposed system. Finally, I thank my family and colleagues for their continued support during the development process.

Executive Summary

This report presents the current development status of Component 1 of PaddyGuard AI: a Sinhala voice-based rice disease diagnosis pipeline intended to make digital crop diagnosis more accessible to Sri Lankan paddy farmers. The component accepts a farmer's spoken Sinhala symptom description, performs automatic speech recognition (ASR), converts the transcript into features for natural-language disease classification, applies uncertainty and out-of-distribution (OOD) checks, asks follow-up questions when confidence is insufficient, and returns diagnostic information with Sinhala text-to-speech support.

The current demonstrable implementation uses a React/Vite frontend, a FastAPI API gateway, a lightweight Railway-hosted voice-NLP service, and a separately hosted GPU ASR service on Modal. The ASR service uses the exact Hugging Face checkpoint Lingalingeswaran/whisper-small-sinhala. Disease classification is currently performed using TF-IDF features with a probabilistic Support Vector Machine classifier over four target categories: Bacterial Blight, Leaf Blast, Brown Spot, and Healthy. The deployment has been functionally tested through the public Gateway, including a successful Brown Spot request that returned HTTP 200, a disease prediction, confidence score, and a request for follow-up because confidence was below the current implementation threshold.

The report deliberately separates completed engineering work from research targets. Formal WER/CER evaluation, comparison of TF-IDF+SVM against BERT/RoBERTa, calibrated OOD evaluation, TTS Mean

Opinion Score testing, and full user evaluation remain research activities to be completed before final claims can be made.

Table of Contents

1. Introduction 1
1.1 Background 1
1.2 Research Problem and Gap 2
1.3 Scope of This Development Report 2
2. Objectives and Research Questions 3
2.1 Main Objective 3
2.2 Specific Objectives 3
2.3 Research Questions 4
3. Development Methodology 5
3.1 Research and Engineering Approach 5
3.2 Iterative Development Strategy 5
3.3 Research Integrity and Evaluation Boundaries 6
4. System Architecture 7
4.1 PaddyGuard AI Overview 7
4.2 Component 1 Architecture 8
4.3 Production Deployment Architecture 9
4.4 API Integration 10
5. Component 1 Implementation 11
5.1 Frontend Voice Capture 11
5.2 ASR Module 12
5.3 NLP Classification 13
5.4 OOD Detection and Confidence Handling 14
5.5 Adaptive Follow-up Dialogue 15
5.6 Text-to-Speech 15
6. Cloud Deployment and Integration 16
6.1 Railway Services 16
6.2 Modal ASR Service 17
6.3 Security and Configuration 18
7. Testing and Current Results 19
7.1 Functional Testing 19
7.2 ASR Smoke Tests 20
7.3 End-to-End Gateway Test 20
7.4 Interpretation and Limitations 21
8. Challenges and Resolutions 22
9. Current Status and Remaining Work 23
10. Evaluation Plan 24
11. Conclusion 26
References 27
Appendices 29

List of Abbreviations

Abbreviation	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
ASR	Automatic Speech Recognition
CER	Character Error Rate
CORS	Cross-Origin Resource Sharing
DSR	Design Science Research
ECE	Expected Calibration Error
HTTP	Hypertext Transfer Protocol
ML	Machine Learning
NLP	Natural Language Processing
OOD	Out-of-Distribution
SVM	Support Vector Machine
TF-IDF	Term Frequency-Inverse Document Frequency
TLS	Transport Layer Security
TTS	Text-to-Speech
WER	Word Error Rate

1. Introduction

1.1 Background

Rice is a strategically important crop in Sri Lanka, and timely identification of crop disease is important for reducing avoidable yield loss. The PaddyGuard AI research project investigates a multimodal decision-support platform that combines voice-based symptom understanding, rice leaf image classification, pest detection, and treatment/advisory support. The voice component addresses an accessibility problem that is not solved by image-only systems: a farmer may be able to describe visible symptoms in Sinhala even when a suitable leaf image cannot be captured.

The approved proposal defines the voice component as a Sinhala-first pipeline that converts spoken symptom descriptions into text, interprets symptoms using NLP, predicts one of four target states, and returns the result through text and synthesized speech. The four current target classes are Bacterial Blight, Leaf Blast, Brown Spot, and Healthy. The component is also designed to participate in the wider PaddyGuard AI architecture by handing diagnostic information to the other services.

1.2 Research Problem and Gap

The proposal identifies three practical limitations that motivate this component: dependence on clear image input, limited Sinhala voice interaction in agricultural diagnosis tools, and the risk of forcing unclear or unrelated symptom descriptions into a known disease class. The research therefore explores whether Sinhala natural-language symptom descriptions can provide useful disease evidence, how traditional text classification compares with transformer-based approaches, and how uncertainty-aware dialogue can improve the reliability of voice-only diagnosis.

Development focus

The current report concentrates on the engineering artefact and verified integration progress. Claims about final diagnostic accuracy, WER/CER, usability, or superiority over alternative models are intentionally deferred until formal experiments are completed.

1.3 Scope of This Development Report

This document records the current implementation of Component 1, the production-style deployment that has been established for demonstration and integration testing, observed smoke-test results, technical challenges encountered, and the remaining research evaluation work. It is not a final dissertation and should not be interpreted as evidence that every numerical target in the proposal has already been achieved.

2. Objectives and Research Questions

2.1 Main Objective

To design, develop, and evaluate a Sinhala voice-based symptom understanding and NLP-driven rice disease classification system that enables Sri Lankan paddy farmers to receive accessible disease diagnoses and that can integrate with the image, pest, and treatment/advisory components of PaddyGuard AI.

2.2 Specific Objectives

1. Develop a Sinhala agricultural symptom dataset suitable for ASR and disease-classification experiments, with the proposal targeting at least 1,500 labelled utterances.

2. Implement and evaluate Sinhala ASR on agricultural speech, with formal WER and CER measurement on a held-out test set.
3. Develop and compare a traditional TF-IDF+SVM baseline against transformer-based text classifiers such as BERT and RoBERTa/XLM-R.
4. Implement uncertainty handling and adaptive follow-up questions for low-confidence diagnoses.
5. Provide Sinhala speech output for predictions and follow-up prompts, and evaluate user-perceived speech quality.
6. Integrate the voice component with the central PaddyGuard API layer and the other project components.

2.3 Research Questions

- Can natural-language symptom descriptions accurately identify the selected rice disease categories?
- How does a transformer-based NLP classifier compare with a traditional TF-IDF+SVM baseline for this domain?
- How reliable is the voice-based diagnostic pathway compared with image-based disease detection, and when should multimodal confirmation be preferred?

3. Development Methodology

3.1 Research and Engineering Approach

The research follows a Design Science Research orientation combined with experimental machine-learning evaluation. Design Science is appropriate because the work produces and iteratively refines a working artefact: a farmer-facing Sinhala voice diagnostic service. Experimental evaluation is required because the research questions depend on measurable comparison of ASR and NLP models rather than implementation alone.

The development work is organized as an incremental pipeline. Each processing stage can be tested independently before integration: audio capture, ASR, transcript processing, classification, OOD/confidence handling, follow-up dialogue, TTS, and API orchestration. This separation has been especially useful during cloud deployment because resource-intensive ASR can be isolated from the lightweight NLP/API service.

3.2 Iterative Development Strategy

Iteration	Primary goal	Current outcome
I1	Establish web/API skeleton and voice flow	React/Vite frontend, FastAPI gateway and voice service implemented.
I2	Create executable NLP baseline	TF-IDF vectorizer and probabilistic SVM model integrated.
I3	Add reliability controls	OOD heuristics, confidence/margin checks, follow-up sessions and question bank implemented.
I4	Stabilize cloud deployment	Voice service moved to lightweight hosted-ASR adapter; health checks no longer load Whisper.
I5	Restore Sinhala-specialized ASR	Exact Sinhala Whisper checkpoint deployed separately on Modal and connected over HTTPS.

Iteration	Primary goal	Current outcome
I6	End-to-end verification	Gateway-to-voice-to-Modal path returned HTTP 200 and a valid disease response.

3.3 Research Integrity and Evaluation Boundaries

A central requirement for this development report is to distinguish implementation status from research evidence. For example, the proposal states targets such as WER no greater than 15%, classification accuracy of at least 85%, a follow-up threshold of 0.70, TTS MOS of at least 3.5, and an eight-second voice-to-prediction target. These remain evaluation targets unless supported by a completed experiment. The current code also uses thresholds that differ from the proposal in places; those differences must be reconciled and justified before final evaluation.

4. System Architecture

4.1 PaddyGuard AI Overview

PaddyGuard AI is implemented as a modular web platform. A React/Vite frontend communicates with a central FastAPI Gateway. The Gateway routes requests to independently deployable services for voice/NLP diagnosis, leaf disease analysis, pest detection, treatment/advisory functions, and user management. This microservice-oriented structure allows each research component to evolve independently while retaining a single user-facing application.

Component	Responsibility	Primary implementation
Frontend	Farmer/user interface, voice capture, result presentation	React, Vite, TypeScript
Gateway	Single public API entry point and routing	FastAPI
Voice-NLP (Component 1)	ASR orchestration, NLP classification, OOD, follow-up, TTS	FastAPI, scikit-learn, hosted Whisper
Leaf disease	Image-based rice disease classification and OOD	FastAPI, PyTorch
Pest detection	Pest detection and unknown-input handling	FastAPI, YOLO-based pipeline
Treatment/advisory	Treatment recommendations and agricultural chatbot	Node.js/Express, RAG
User management	Authentication and persistence	FastAPI, PostgreSQL

4.2 Component 1 Architecture

The Component 1 request path is sequential but uncertainty-aware. Audio is captured or uploaded by the browser and submitted to the Gateway. The Gateway forwards the multipart request to the private voice-NLP service. The voice service validates the audio and sends it to the configured ASR provider. The returned Sinhala transcript is passed through the existing translation/normalization stage, then transformed by the TF-IDF vectorizer and classified by the SVM. Reliability checks evaluate disease vocabulary, confidence, entropy and score margin. If the result is considered OOD, the system returns an unknown/clarification

response. If it is in-distribution but below the follow-up threshold, a dialogue session is created and a discriminating question is returned. Otherwise the disease result proceeds to TTS and the API response.

Component 1 processing flow

Browser audio → Gateway → voice-nlp → hosted Sinhala ASR → Sinhala transcript → translation/normalization → TF-IDF → SVM → OOD/confidence logic → follow-up or diagnosis → Sinhala TTS → browser.

4.3 Production Deployment Architecture

The current demonstration deployment separates CPU-light API work from GPU ASR inference. The public Gateway and private voice-NLP service are hosted on Railway. The voice-NLP service calls a protected HTTPS endpoint on Modal, where the Sinhala-specialized Whisper checkpoint is loaded on GPU only when required. This design avoids loading the speech model inside the constrained Railway voice container while preserving the same downstream classification pipeline.

Layer	Deployment	Exposure
Frontend	Planned Vercel deployment	Public web application
Gateway	Railway	Public HTTPS
voice-nlp	Railway	Private within Railway project
user-management	Railway	Private within Railway project
PostgreSQL	Railway	Private persistent service
ASR inference	Modal	Public HTTPS endpoint protected by application-level Bearer authentication
Leaf/Pest/Treatment	Separate team deployments	Public service URLs when hosted by other project members

4.4 API Integration

Route	Purpose	Current status
POST /api/v1/voice/diagnose	Upload audio and obtain transcript/diagnosis	Implemented and end-to-end tested
POST /api/v1/voice/followup	Submit answer for an active low-confidence dialogue session	Implemented; further hosted UI testing pending
POST /api/v1/auth/login	Authenticate user through Gateway	Implemented in wider platform
POST /api/v1/chat/message	Advisory/chat request	Wider platform integration route

The voice frontend sends the audio in multipart form-data using the field name audio. The voice-NLP hosted ASR adapter separately sends a multipart file field to the Modal ASR endpoint. This distinction is intentional: the browser contract is with the Gateway, while the internal ASR provider contract is between voice-NLP and Modal.

5. Component 1 Implementation

5.1 Frontend Voice Capture

The voice diagnosis page supports recording or uploading common audio formats and submits the recording to the Gateway. The interface is designed around a Sinhala-first workflow and displays the predicted disease, confidence, follow-up state, OOD warnings and diagnostic history. The frontend uses a configurable Vite API base URL so production deployment can target the public Gateway without embedding private service addresses in browser code.

5.2 Automatic Speech Recognition Module

Early deployment work used a generic hosted ASR provider with whisper-large-v3. Functional integration succeeded, but a manually checked Sinhala recording was transcribed incorrectly. During local development, a Sinhala-specialized Whisper checkpoint had performed better on the same type of speech. To preserve the specialized model without forcing Railway to host a memory-intensive Transformer, the exact checkpoint Lingalingeswaran/whisper-small-sinhala was deployed as an isolated Modal service.

The Modal service uses a persistent Hugging Face model cache, scale-to-zero behavior, a conservative container limit, and a protected /listen endpoint. It accepts an audio file and compatible form metadata, but the server pins the model internally so a caller cannot select an arbitrary model. The service returns JSON containing the Sinhala transcript, model identifier, language and inference duration.

Important research distinction

The currently deployed Sinhala-specialized model is not evidence that the proposal's whisper-large-v3 target has been met. The final study should compare candidate ASR configurations on the same labelled test recordings using WER/CER.

5.3 NLP Disease Classification

The executable classifier currently uses TF-IDF text features and a probabilistic SVM. Two serialized model artefacts are loaded by the voice service: the TF-IDF vectorizer and the trained disease classifier. The output label mapping is Bacterial Blight, Leaf Blast, Brown Spot and Healthy. Class probabilities are used by the reliability layer and are also returned in the diagnostic response.

Label ID	Class
0	Bacterial Blight
1	Leaf Blast
2	Brown Spot
3	Healthy

The transformer comparison described in the proposal (BERT and RoBERTa/XLM-R) remains part of the research evaluation plan. The current deployed demo should therefore be described as the TF-IDF+SVM baseline implementation, not as the final selected research model.

5.4 OOD Detection and Confidence Handling

The current reliability layer combines several heuristics rather than relying on classifier confidence alone. It checks whether the input is sufficiently long, whether disease-related vocabulary is present, whether the

input matches blocked/non-disease topics, the classifier's top probability, prediction entropy, and the margin between the highest two class probabilities. This is intended to reduce the risk of returning a confident disease label for irrelevant or poorly recognized speech.

Current implementation parameter	Value
OOD threshold	0.50
Follow-up threshold	0.75
Entropy threshold	1.20
Top-1 / Top-2 margin threshold	0.10
Minimum text words	3
Minimum symptom words	2

The proposal specifies a follow-up threshold of 0.70, while the current implementation uses 0.75. This difference is recorded as an implementation decision that must be experimentally calibrated and reconciled in the final methodology.

5.5 Adaptive Follow-up Dialogue

When a request is considered in-distribution but the confidence is below the follow-up threshold, the service starts a dialogue session and returns a disease-discriminating question. The question bank contains bilingual Sinhala/English prompts. User answers are mapped to symptom hints and the classification can be refined over a maximum of three questions. Session state is stored through Redis, allowing the follow-up endpoint to continue the diagnostic interaction across HTTP requests.

5.6 Text-to-Speech Output

The voice response payload includes synthesized audio so that the diagnosis or clarification can be delivered without requiring the user to read the screen. TTS is part of the accessibility goal of the research. At this stage, functional generation is present, but formal Mean Opinion Score testing with native Sinhala speakers has not yet been completed and should not be reported as achieved.

6. Cloud Deployment and Integration

6.1 Railway Services

The user's Railway project currently hosts the public Gateway, the private voice-NLP service, user management and PostgreSQL. The voice service is intentionally unexposed to the public internet; the Gateway reaches it through Railway's internal network. The Gateway is the stable browser-facing API origin. This separation reduces the number of publicly reachable application services and keeps internal service discovery out of frontend code.

An earlier version of voice-NLP attempted to load Whisper eagerly during application startup. On a small Railway container this led to repeated restarts and health-check failure, strongly indicating that the ASR model was unsuitable for that memory budget. The service was refactored so health checks remain lightweight and ASR can be provided externally. This allowed the classifier and API service to start reliably.

6.2 Modal ASR Service

Modal was selected for the exact Sinhala Whisper checkpoint because it allows the ASR model to run separately from Railway and scale down when idle. The service uses a persistent model cache under /model-cache, initializes the model once per warm container, and exposes an application endpoint at /listen. The endpoint enforces a shared Bearer credential stored using Modal's secret mechanism. Upload validation limits supported formats and maximum payload size before GPU inference is invoked.

ASR service property	Current implementation
Model	Lingalingeswaran/whisper-small-sinhala
Language	si
Endpoint method	POST
Endpoint route	/listen
Authentication	Bearer token, application-level validation
Maximum upload size	25 MiB
Supported formats	OGG, WAV, MP3, M4A, WebM
Scale-down window	180 seconds
Warm minimum	None configured

6.3 Security and Configuration

Production credentials are not stored in source code. The shared ASR Bearer value is stored independently in Modal and Railway environment configuration. The browser does not receive this credential because only the server-side voice-NLP service calls Modal. Similarly, the frontend should communicate only with the public Gateway and should never receive Railway private service addresses. CORS should be restricted to the final deployed frontend origin once the Vercel domain is known.

- Do not commit .env files containing production credentials.
- Use HTTPS for public Gateway and Modal communication.
- Keep voice-NLP and database services private where possible.
- Use exact production frontend origins in CORS rather than a wildcard.
- Treat raw voice recordings as ephemeral processing data unless explicit research consent and retention procedures are established.

7. Testing and Current Results

7.1 Functional Testing

Testing to date has focused on engineering correctness and integration. Unit tests were added around the hosted ASR adapter and the Modal HTTP boundary. The Modal endpoint tests cover Bearer authentication, request compatibility, success response structure, pinned model behavior, unsupported/oversized uploads and error hygiene. Existing hosted-ASR adapter tests were also retained to ensure that the provider abstraction remains compatible.

Test area	Observed result
Modal endpoint compile check	Passed
Modal HTTP endpoint unit tests	7 passed
voice_nlp hosted-ASR adapter tests	20 passed
Direct Modal HTTPS audio request	HTTP success with correct manually verified Sinhala transcript
Gateway → voice-nlp → Modal request	HTTP 200 with diagnosis response
Railway voice-nlp deployment	Online after hosted-ASR refactor

7.2 ASR Smoke Tests

Two manually checked recordings were used as smoke tests for the exact Sinhala checkpoint. These are not a formal test set and therefore cannot be used to claim a WER value or general accuracy. They are useful for verifying that the deployed model can reproduce known transcripts and that the HTTP boundary does not alter the inference behavior.

Recording	Observed transcript	Inference time	Interpretation
Sample 1	ගොයම් කොලාග කහපාටට හැරිලා	7.791 s	Manually confirmed correct
Sample 2	ඩයමන්ඩ් හැඩතියි පුල්ලි තියෙනවා.	8.357 s via HTTPS test	Manually confirmed correct

The same second sample had previously produced an incorrect Sinhala transcript when sent through the generic whisper-large-v3 hosted provider. This is qualitative evidence that model choice matters for this use case, but a controlled WER/CER experiment over a representative dataset is required before drawing a research conclusion.

7.3 End-to-End Gateway Test

A Brown Spot audio file was submitted through the public Gateway Swagger interface to POST /api/v1/voice/diagnose after Railway was configured to use the Modal endpoint. The request completed with HTTP 200. The response predicted Brown Spot (label 2) with confidence 0.5649, marked the input as in-distribution, and set needs_followup to true. This behavior is consistent with the current 0.75 follow-up threshold.

Field	Observed value
HTTP status	200
Disease	Brown Spot
Label ID	2
Confidence	0.5649
is_ood	false
needs_followup	true
Status	Low confidence - ask follow-up

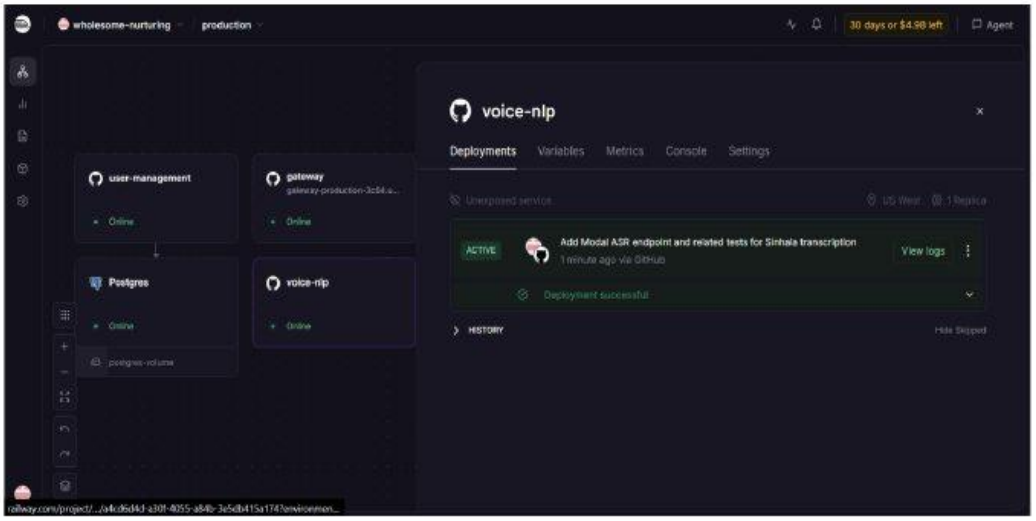


Figure 1. Railway voice-NLP deployment showing the service online and deployment successful.

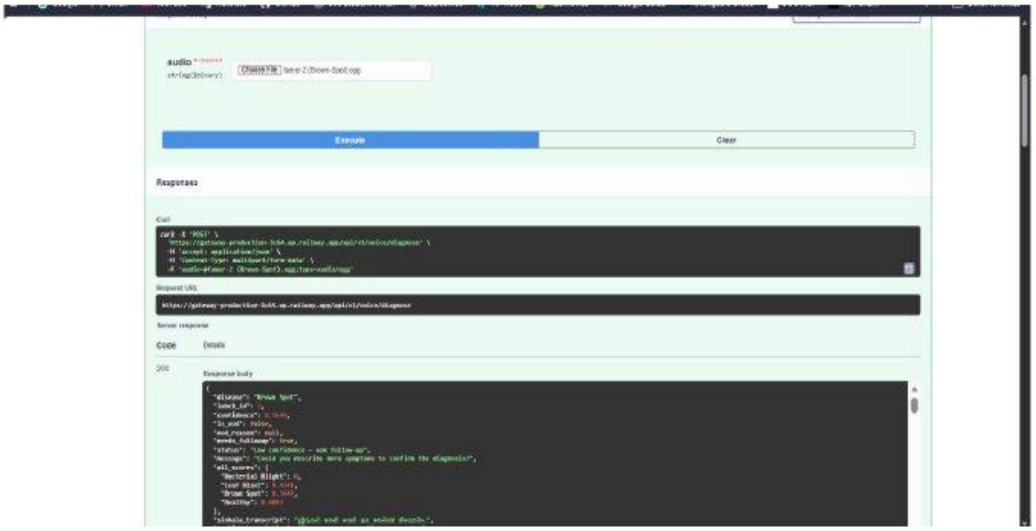


Figure 2. Gateway Swagger end-to-end voice diagnosis test returning HTTP 200 and Brown Spot output.

7.4 Interpretation and Limitations

The current results demonstrate functional integration, not final research performance. The system has shown that a browser/API request can traverse the Gateway, private voice service, authenticated Modal ASR endpoint and NLP pipeline, then return a structured diagnosis. However, a small number of successful recordings cannot establish ASR accuracy, disease-classification accuracy, OOD reliability, calibration quality or user usability. Those claims require a pre-defined evaluation dataset, reference transcripts and appropriate statistical metrics.

8. Challenges and Resolutions

Challenge	Impact	Resolution / current approach
Azure student subscription unavailable	Could not use planned Azure hosting	Moved deployment experiments to

Challenge	Impact	Resolution / current approach
		alternative providers.
Oracle A1 capacity unavailable	ARM VM could not be provisioned	Abandoned this route rather than depending on unavailable capacity.
Railway ASR startup resource pressure	Voice service repeatedly restarted during eager Whisper load	Refactored ASR into local/hosted providers and kept Railway service lightweight.
Generic hosted ASR produced poor Sinhala transcript on checked sample	Downstream translation/classification became unreliable	Deployed exact Sinhala-specialized Whisper checkpoint on Modal.
Modal volume mount conflict at /cache	Initial Modal run failed	Moved persistent cache to /model-cache.
Whisper generation length conflict	Generation raised max target position error	Set max_new_tokens to 256 and adjusted runtime generation configuration.
GPU container preemption	One smoke test restarted	Modal restarted the function with the same input; subsequent inference completed.
Proposal/runtime threshold mismatch	Methodology could become inconsistent	Recorded mismatch; final threshold will be calibrated and justified experimentally.

These issues influenced the final development architecture. In particular, the separation of ASR from the Railway API service was not only a deployment workaround; it also creates a cleaner experimental boundary for comparing ASR models while keeping the NLP classifier and public API constant.

9. Current Status and Remaining Work

Work item	Status	Notes
React voice diagnosis UI	Implemented	Production hosting on Vercel is the next deployment task.
Gateway voice routes	Implemented	Public Railway Gateway is functioning.
Railway voice-NLP service	Implemented / deployed	Private service is online.
TF-IDF + SVM baseline	Implemented	Current executable classifier.
OOD heuristic layer	Implemented	Needs formal OOD evaluation and calibration.
Adaptive follow-up logic	Implemented	Hosted end-to-end follow-up UI testing still required.
Sinhala TTS	Implemented functionally	MOS/usability evaluation pending.
Exact Sinhala Whisper ASR	Deployed on Modal	Two manually verified smoke tests successful.
Formal ASR WER/CER evaluation	Pending	Requires reference transcript test set.

Work item	Status	Notes
BERT/RoBERTa comparison	Pending	Required for research comparison objective.
Voice vs image / multimodal comparison	Pending	Requires agreed evaluation protocol across components.
Frontend production deployment	In progress	Vercel planned; Gateway CORS must be set to final origin.
Final performance/usability study	Pending	Must be completed before final claims.

9.1 Immediate Next Development Steps

7. Deploy the React/Vite frontend to Vercel using the public Railway Gateway as VITE_API_URL.
8. Update Gateway/related CORS configuration to allow the exact Vercel production origin.
9. Test voice recording/upload and the complete follow-up flow from the hosted browser UI.
10. Create a controlled ASR evaluation set with reference Sinhala transcripts and speaker/session separation.
11. Run WER/CER comparison for candidate ASR configurations on the same test set.
12. Train/evaluate BERT and RoBERTa/XLM-R alternatives using leakage-safe train/validation/test splits.
13. Calibrate classifier and OOD thresholds using validation data, then freeze thresholds before final testing.
14. Conduct TTS MOS/usability testing and document latency/resource behavior.

10. Research Evaluation Plan

10.1 Dataset and Split Discipline

The final evaluation should prevent speaker and augmentation leakage. Recordings from the same speaker/session should not be distributed across training and test sets in a way that inflates performance. Any augmented versions of a source recording must remain within the same split as the original. A held-out test set should be frozen before final model comparison.

10.2 ASR Evaluation

Metric	Purpose
WER	Measures word-level transcription errors against reference Sinhala transcripts.
CER	Useful for Sinhala script and tokenization variation; measures character-level error.
Latency	Measures inference and end-to-end response time under defined conditions.
Error analysis	Categorizes substitutions, deletions, insertions and domain-specific symptom failures.

10.3 Disease Classification Evaluation

TF-IDF+SVM, BERT and RoBERTa/XLM-R should be evaluated on the same fixed test examples. Report accuracy together with macro-averaged precision, recall and F1 so class imbalance does not disappear behind a single accuracy figure. A confusion matrix should be used to identify clinically/agronomically important confusion patterns among Bacterial Blight, Leaf Blast, Brown Spot and Healthy.

10.4 OOD and Calibration Evaluation

OOD evaluation should include genuinely unrelated speech/text and symptom descriptions outside the four trained categories. Recommended metrics include AUROC, AUPR and FPR95. Confidence calibration should be assessed using Expected Calibration Error and Brier score, with temperature scaling or another calibration method considered only on validation data. The final OOD/follow-up thresholds should be chosen before evaluating the frozen test set.

10.5 Adaptive Dialogue Evaluation

The follow-up mechanism should be evaluated for more than completion rate. Useful measures include confidence gain after each question, percentage of initially ambiguous cases corrected, number of questions required, and error rate introduced by misleading or misunderstood answers. The maximum-three-question rule should also be assessed for usability.

10.6 User and TTS Evaluation

A Sinhala-speaking user study should evaluate whether the interface is understandable and practical in realistic conditions. TTS can be assessed with a Mean Opinion Score protocol, while broader usability can include task completion, perceived clarity, confidence in the result, and qualitative feedback. Participant counts, recruitment, consent and analysis procedures should follow the approved ethics process.

10.7 Deployment Evaluation

Production evaluation should measure end-to-end latency from browser upload to returned diagnosis, cold-start behavior of the scale-to-zero ASR service, failure handling, and low-bandwidth behavior. The proposal's performance and availability numbers should remain stated as targets until measured under a documented load-test protocol.

11. Conclusion

Component 1 of PaddyGuard AI has progressed from proposal-level architecture to a working, cloud-integrated prototype. The current implementation provides Sinhala audio ingestion, hosted Sinhala ASR, TF-IDF+SVM disease classification, OOD/confidence handling, adaptive follow-up logic, TTS output and Gateway-based integration. A key engineering achievement is the separation of the resource-intensive Whisper model from the lightweight Railway voice service, allowing the exact Sinhala-specialized checkpoint to run on Modal while the public application continues to use a stable Gateway contract.

The present evidence supports a conclusion of functional feasibility and successful system integration. It does not yet support final claims about WER, classification accuracy, OOD effectiveness, user acceptance or superiority over alternative models. The next phase should therefore prioritize controlled experiments, threshold calibration, hosted frontend testing and user evaluation. Maintaining this distinction between a working prototype and validated research results will strengthen the final dissertation and make the eventual conclusions defensible.

References

[1] Food and Agriculture Organization of the United Nations, "FAOSTAT - Crops and Livestock Products," FAO, Rome, Italy.

[2] R. C. Nayak, P. K. Singh, and S. K. Roy, "Diseases of rice and their management," in Rice Production in India: Under Changing Climatic Conditions, Springer, 2023, pp. 315-342.

[3] Hector Kobbekaduwa Agrarian Research and Training Institute, "Survey on Digital Technology Adoption among Smallholder Paddy Farmers in Sri Lanka," Research Rep. No. 174, 2021.

[4] S. P. Mohanty, D. P. Hughes, and M. Salathe, "Using deep learning for image-based plant disease detection," Frontiers in Plant Science, vol. 7, p. 1419, 2016.

[5] A. Radford et al., "Robust speech recognition via large-scale weak supervision," in Proc. 40th Int. Conf. Machine Learning, 2023, pp. 28492-28518.

[6] D. Povey et al., "The Kaldi speech recognition toolkit," in Proc. IEEE ASRU, 2011.

[7] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in Proc. NAACL-HLT, 2019, pp. 4171-4186.

[8] Y. Liu et al., "RoBERTa: A robustly optimized BERT pretraining approach," arXiv:1907.11692, 2019.

[9] P. K. Sethy et al., "Deep feature-based rice leaf disease identification using support vector machine," Computers and Electronics in Agriculture, vol. 175, p. 105527, 2020.

[10] T. Barnett, "Interactive voice response systems for agricultural advisory services: A review of evidence from sub-Saharan Africa and South Asia," Journal of Development Effectiveness, vol. 12, no. 4, pp. 293-310, 2020.

[11] V. Singh and A. K. Misra, "Detection of plant leaf diseases using image segmentation and soft computing techniques," Information Processing in Agriculture, vol. 4, no. 1, pp. 41-49, 2017.

[12] J. Howard and S. Ruder, "Universal language model fine-tuning for text classification," in Proc. ACL, 2018, pp. 328-339.

[13] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in Proc. NeurIPS, 2020, pp. 9459-9474.

[14] H. Touvron et al., "Llama 2: Open foundation and fine-tuned chat models," arXiv:2307.09288, 2023.

[15] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in Proc. CVPR, 2016, pp. 770-778.

Appendix A - Current Component 1 API Contract

Operation	Request	Key response fields
Diagnose	POST /api/v1/voice/diagnose; multipart audio	disease, label_id, confidence, is_ood, needs_followup, sinhala_transcript, english_translation, session_id, followup_question, tts_audio_b64
Follow-up	POST /api/v1/voice/followup; JSON answer + session_id	updated diagnosis/confidence, next question or final result

Appendix B - Current Deployment Configuration (Non-secret)

Variable	Production intent / value
ASR_BACKEND	hosted
HOSTED_ASR_MODEL	Lingalingeswaran/whisper-small-sinhala
HOSTED_ASR_LANGUAGE	si

Variable	Production intent / value
HOSTED_ASR_RESPONSE_FORMAT	json
HOSTED_ASR_TIMEOUT_SECONDS	60
HOSTED_ASR_TEXT_FIELD	text
MODEL_PATH	models/paddyguard_best_classifier.pkl
TFIDF_PATH	models/paddyguard_tfidf.pkl

Secret values such as API keys, Redis URLs and JWT secrets are intentionally excluded from this report.

Appendix C - Development Evidence Summary

- Railway voice-NLP service successfully deployed and online after the hosted-ASR refactor.
- Modal HTTPS endpoint successfully returned the expected Sinhala transcript for a manually verified sample.
- Gateway Swagger test successfully traversed Gateway → private voice-NLP → Modal ASR → NLP pipeline and returned HTTP 200.
- Brown Spot test response returned confidence 0.5649 and correctly activated the low-confidence follow-up state under the current threshold.
- Frontend production deployment to Vercel is the next integration activity.

Appendix D - Proposed Final Experiment Checklist

Experiment	Required output
ASR comparison	WER, CER, latency, error analysis by model and recording condition
NLP model comparison	Accuracy, macro precision/recall/F1, confusion matrix
OOD evaluation	AUROC, AUPR, FPR95, threshold selection
Calibration	ECE, Brier score, reliability diagram
Dialogue evaluation	confidence gain, correction rate, questions per case
TTS/user study	MOS and usability/task feedback
System test	end-to-end latency, cold-start behavior, failure cases, bandwidth observations

3. Component 2 — Rice Leaf Disease Classification

Original report reproduced in full, without summarization or omission.

PaddyGuard AI – Component 2: Development and Deployment Report

Explainable Rice Leaf Disease Classification with Out-of-Distribution Detection and Human-in-the-Loop Active Learning

Hewanayake H. M. L. M (IT22168740) · Project ID R26-SE-015

October 2026

Supervisor: Dr. Nathali Silva · Co-supervisor: Ms. Poorna Panduwawala

B.Sc. (Hons) in Information Technology, Specialising in Software Engineering · Sri Lanka Institute of Information Technology

Declaration

I declare that this report documents my own work on Component 2 of PaddyGuard AI (Rice Leaf Disease Classification with Out-of-Distribution Detection). It covers the model, the Flask inference service, the expert-review and gated fine-tuning workflow, the Railway cloud deployment and the integration with the PaddyGuard gateway. Results described as completed come from experiments, tests and deployments I carried out; limitations and unmet targets are reported as such.

Student: Hewanayake H. M. L. M | Student ID: IT22168740 | Signature: _____ | Date: _____

Supervisor acknowledgement: Dr. Nathali Silva | Signature: _____ | Date: _____

Acknowledgement

I thank my supervisor, Dr. Nathali Silva, and my co-supervisor, Ms. Poorna Panduwawala, for their guidance and feedback, and the external domain expert, Mrs. Anoma Gunarathna, for sharing her knowledge of rice cultivation in Sri Lanka. I am also grateful to my group members, Weerasinghe K. G. J. P., Kalhara E. P. K. and Keshan B. K., for the integration work that let the components run as one platform, and to my family and friends for their support.

Executive Summary

This report describes the development and cloud deployment of Component 2 of PaddyGuard AI: an explainable rice leaf disease classification service for four classes (Bacterial Blight, Brown Spot, Leaf Blast and Healthy). A fine-tuned EfficientNetB3 reached 97.85% accuracy (macro F1 0.9784) on 838 held-out images, and the deployed active-learning model (Round 2) reached 97.14% using 45% fewer labels. The service adds an energy-based out-of-distribution (OOD) gate, a three-state decision (KNOWN, UNCERTAIN, OOD), Grad-CAM explanations, a weather-aware yield-loss estimate, an expert review queue and a metric-gated fine-tuning pipeline.

The backend is a Flask 3 application served by Gunicorn in a Docker container (Python 3.11, CPU-only PyTorch). It is deployed on Railway as the service **paddyguard-leaf-disease-detection** in the production environment, built automatically from GitHub, online with one replica in US East, and connected to the PaddyGuard FastAPI gateway through the `LEAF_DISEASE_URL` variable.

The report separates what was achieved from what was not. The OOD gate rejected only 52.6% of near-OOD test images at the deployed threshold, below the 85% target (NFR03), because banana and mango leaves resemble rice leaves. Hosted-instance latency and a field study with farmers have not yet been measured.

1. Introduction

1.1 Background and Context

Rice leaf diseases such as Bacterial Blight, Brown Spot and Leaf Blast cause serious yield losses for Sri Lankan smallholders when they are diagnosed late. Deep learning classifiers can recognise these diseases from photographs, but a conventional closed-set classifier returns one of its trained labels for every image, even a photograph of a dog or a banana leaf. PaddyGuard AI is a multimodal platform combining voice-based diagnosis, leaf disease recognition, pest detection and treatment advice. This report covers only Component 2, the leaf disease service.

1.2 Problem Statement and Research Gap

Most published rice disease systems assume that every input belongs to a trained class, rarely explain their decisions, and cannot learn safely from expert corrections. Component 2 addresses three connected problems: unfamiliar inputs that receive confident diagnoses, predictions that users cannot inspect, and models that either never improve or change without evidence. The contribution is the combination of an OOD gate, an uncertainty state, visual explanation and a governed human-in-the-loop update path in one deployed service.

1.3 Scope of This Report

This report describes the dataset and model, the service architecture and API, the reliability features, the Docker and Railway deployment, gateway integration, testing and results, challenges, and remaining work. The full research analysis is in the individual thesis; this report concentrates on engineering and deployment.

2. Objectives and Research Questions

2.1 Main Objective

To build and deploy an explainable rice leaf disease classification service that abstains on unfamiliar or ambiguous images, explains its predictions, and improves from expert feedback only when a held-out test shows no regression.

2.2 Specific Objectives

- Prepare a balanced, field-like four-class dataset with a separate near-OOD probe set.
- Fine-tune an ImageNet-pretrained EfficientNetB3 and reach at least 92% test accuracy (NFR02).
- Reject non-rice images with an energy-based gate (target: at least 85% of OOD images rejected, NFR03).
- Return a three-state decision and a Grad-CAM explanation with an attention-based severity indicator.
- Provide an expert review queue, active-learning batches and metric-gated in-application fine-tuning with backup and rollback.

- Containerise the service, deploy it on Railway with health checks, and integrate it with the PaddyGuard gateway.

2.3 Research Questions

- **RQ1:** How accurately can a transfer-learned CNN classify field-like rice leaf images?
- **RQ2:** How well can post-hoc OOD scores (MSP, energy) separate rice leaves from near-OOD images?
- **RQ3:** Does uncertainty-based active learning use labels more efficiently than random sampling?
- **RQ4:** Can a metric gate make in-application fine-tuning safe?
- **RQ5:** Can the service be deployed and integrated as a maintainable cloud component?

3. Development Methodology

3.1 Research Approach

The work followed a design science approach: build an artefact, evaluate it against stated requirements (FR01–FR12, NFR01–NFR08), and iterate. Model experiments were run in Google Colab on an NVIDIA Tesla T4 GPU. The service was developed locally in Visual Studio Code with Python 3.11, versioned with Git and GitHub, and tested with Postman and automated scripts.

3.2 Development Workflow

Stage	Activity	Output
1. Requirements	Define FR01–FR12 and NFR01–NFR08	Requirements and traceability matrix
2. Data preparation	Build dataset, balance classes, prepare OOD probe set	Train, validation and test folders
3. Model experiments	Train EfficientNetB3, run active learning, tune OOD thresholds	Checkpoints and evaluation outputs
4. Service implementation	Flask blueprints, ModelService, Grad-CAM, repositories	45 Python files, about 5,000 lines
5. Human-in-the-loop layer	Review queue, batches, gated fine-tuning, retention	Training subsystem
6. Integration	Gateway routes, React pages, treatment hand-off	Routes under /api/v1/image
7. Deployment	Dockerise, deploy to Railway, configure variables	Live Railway service
8. Verification	Unit, API and end-to-end tests; evaluation script	24 traced test cases

3.3 Dataset and Image Preprocessing

The dataset combines two public Kaggle collections of smartphone field photographs (Leaf Rice Disease, Indonesia, and Rice Leaf Disease Image), mapped to four classes. The training set is balanced at 1,050 images per class. No locally collected Sri Lankan images were used for the reported results; this is a stated limitation.

Split	Bacterial Blight	Brown Spot	Healthy	Leaf Blast	Total	OOD probe

Split	Bacterial Blight	Brown Spot	Healthy	Leaf Blast	Total	OOD probe
Training	1,050	1,050	1,050	1,050	4,200	–
Validation	225	224	194	194	837	196
Test	225	225	194	194	838	194

The OOD probe set contains banana leaves, mango leaves, bare soil and weeds, chosen as realistic farmer mistakes. No OOD image was used for training. Images are resized to 300 × 300 pixels, converted to tensors and normalised with ImageNet means and standard deviations. Training augmentation (horizontal flip, ±20° rotation, colour jitter ±0.2, random resized crop) is applied only during training.

3.4 Model Development and Selection

The classifier is a torchvision EfficientNetB3 [1] initialised with ImageNet-1K weights, with the head replaced by dropout (p = 0.4) and a linear layer from 1,536 features to four logits. It has 10,702,380 parameters and a 41.4 MB checkpoint. All layers were fine-tuned (AdamW, learning rate 1e-4, batch size 16, up to 15 epochs, class-weighted cross-entropy, seed 42, about 32 minutes on a T4). The checkpoint with the best validation macro F1 (0.9856 at epoch 13) was kept.

EfficientNetB3 was chosen over ResNet50 and DenseNet121 for its accuracy-to-size ratio and its native 300 × 300 input. A ResNet50 baseline built during prototyping was not retained in a form that allows a fair comparison, so a systematic architecture comparison is reported as future work rather than as a result.

The deployed model is the **Round 2 active-learning model** (2,300 labels). Uncertainty sampling reached 97.14% accuracy against 96.66% for random sampling at the same budget.

3.5 Research Integrity and Evidence Boundaries

All classification and OOD figures are measured on held-out sets never used for training or threshold tuning (the thesis notes that Round 2 was chosen using test accuracy, which is discussed there as a threat to validity). Results seen directly in screenshots, database records or evaluation output are marked as passed; behaviour verified only by reading source code is labelled as code inspection.

4. System Architecture

4.1 Position within PaddyGuard AI

The React web application calls the FastAPI gateway, which forwards leaf requests (more than thirty routes under /api/v1/image) to this service together with the Authorization header and multipart bodies. The gateway returns HTTP 503 if the leaf service is unreachable and uses a 120 s timeout. A KNOWN result can open the treatment chatbot (Component 4) with the disease pre-filled.

4.2 Backend Architecture

Layer	Responsibility	Implementation

Layer	Responsibility	Implementation
API layer	Twelve blueprints: analysis, cases, user history, expert review, active learning, fine-tuning, candidates, auth, admin, experts, media, health	Flask 3, Flask-CORS
Model service	Loads checkpoint once, inference, energy score, Grad-CAM, severity, hot reload	PyTorch, torchvision
Risk and weather	Open-Meteo geocoding, archive and forecast; climate score and yield-loss estimate	requests with retries
Repositories	One class per MongoDB collection	PyMongo
Security	Hashed passwords, signed expiring tokens, three roles	Werkzeug, itsdangerous
Training subsystem	Separate process for fine-tuning; backups, candidates and log retention	PyTorch, model_retention.py
Serving	Gunicorn, one worker, prediction lock	Docker, Railway

4.3 Analysis Data Flow

Upload (with city or GPS and field details) → validation and duplicate check (SHA-256, five-minute window) → decode and preprocess → EfficientNetB3 logits → energy score and softmax confidence → three-state decision → for KNOWN results: Grad-CAM, severity, weather and yield-loss risk → images stored in Cloudinary and case stored in MongoDB → JSON response. OOD uploads are deleted and nothing is stored.

4.4 API Design

Method and path	Purpose	User
POST /api/analyze	Analyse an image with field details	Any (token optional)
GET /api/cases, DELETE /api/cases/{id}	List or delete cases	User
PATCH /api/cases/{id}/feedback	Record farmer feedback	User
POST /api/cases/{id}/refresh-weather	Refresh weather and risk	User
GET /api/user/history	Personal history and analytics	User
GET /api/expert/review-queue	Uncertain cases awaiting review	Expert
POST /api/expert/review-queue/{id}/verify	Submit verified label	Expert
POST /api/expert/active-learning/prepare-batch	Create a batch of verified samples	Super Admin
GET /api/expert/fine-tune/readiness	Check samples and test set	Super Admin
POST /api/expert/fine-tune/start	Launch a fine-tuning job	Super Admin
POST /api/expert/fine-tune/{job}/promote	Promote an eligible candidate	Super Admin
POST /api/expert/model-candidates/upload	Upload an external candidate	Super Admin
GET /health	Service, database and model status	Public

Through the gateway these routes are exposed under /api/v1/image. Errors use standard codes (400, 401, 403, 404, 429, 500).

4.5 External Interfaces

System	Purpose	Failure handling
MongoDB Atlas	Cases, users, jobs, deployments	/health reports status; requests fail with 500 and are logged
Cloudinary	Original images and Grad-CAM overlays	Training falls back to local copies
Open-Meteo	Geocoding, 7-day archive, 3-day forecast	Three attempts with back-off; clear message if city not found
PaddyGuard gateway	Single entry point	Returns 503 if unreachable

5. Component Implementation

5.1 Model Service

`ModelService` is created once at start-up. It builds the EfficientNetB3 architecture, loads the checkpoint named by `MODEL_PATH` (a raw state dictionary or a wrapped checkpoint), selects the available device, switches to evaluation mode and registers Grad-CAM hooks on `features[-1]`. `predict` returns the status, predicted class, confidence, energy score, all four class probabilities, review flags, severity and, for KNOWN results, a base64 Grad-CAM overlay. `reload` loads a promoted model without restarting the server.

5.2 Energy-Based OOD Gate

The energy score is computed from the logits [2]. An image whose score falls below the threshold (3.0 in deployment) is rejected as OOD. On the test sets the energy and MSP scores both reached an AUROC above 0.95, but neither separates near-OOD images cleanly (Section 7.4). The energy gate was adopted because it gave the lowest false-rejection rate and the highest system accuracy for the initial model.

5.3 Three-State Decision

- **OOD:** energy below 3.0. Message: “Unknown image (not a recognised rice leaf)”. No diagnosis, heatmap or risk; file deleted; nothing stored.
- **UNCERTAIN:** passes the gate but confidence below 0.70. The class is returned with a warning, no Grad-CAM or severity. Confidence below 0.50 also flags the case for expert review (LOW_CONFIDENCE).
- **KNOWN:** otherwise. Grad-CAM, severity and yield-loss risk are computed.

5.4 Grad-CAM and Severity Proxy

Grad-CAM [3] weights the 1,536 activation maps of the last convolutional block by the spatial mean of their gradients, applies ReLU, resizes to 300 × 300 and blends the JET-coloured map with the image (60% image, 40% heatmap). Severity is the percentage of pixels with a normalised Grad-CAM value of at least 0.55, mapped to Negligible, Mild, Moderate, Severe or Critical. The interface labels it an attention visualisation and severity proxy, not a lesion mask, and the API reports the method name `gradcam_attention_proxy`.

5.5 Expert Review and Gated Fine-Tuning

Cases with low confidence are selected by Top-K uncertainty for expert review. Verified cases become eligible training samples (OOD-labelled, unverified, empty-label and already-consumed cases are excluded). A Super Admin can export batches or start an in-application fine-tuning job (minimum three eligible samples, three epochs: two head-only and one full). The trainer runs as a separate process, so inference is never blocked. A candidate is promoted only if it beats the deployed model on the held-out test set; promotion creates a backup, replaces the active model, hot-reloads it and records a deployment record. Failed or regressing candidates leave the serving model untouched.

5.6 Weather-Aware Yield-Loss Risk

For KNOWN results the service fetches recent and forecast weather for the field location, computes a climate score and estimates loss as a percentage and, when expected yield is supplied, in kilograms, with an expandable calculation breakdown.

5.7 Security and Error Handling

Passwords are salted hashes, tokens are signed and expire after 8 hours, and three roles (farmer, expert, Super Admin) are enforced on every protected route. Duplicate uploads within five minutes return HTTP 429. Input validation returns HTTP 400 before any upload where possible. Credentials are read from environment variables and are never stored in the repository.

6. Cloud Deployment and Integration

6.1 Railway Deployment Model

The leaf service was moved from the team repository into a dedicated GitHub repository and deployed on Railway as **paddyguard-leaf-disease-detection** in the production environment. Railway builds the Docker image on every push and replaces the running container only if the new deployment passes its health check on `/health`, so a broken build never replaces a working one. The dashboard shows the following state.

Item	Observed in Railway dashboard
Project / environment	valiant-optimism / production
Service	paddyguard-leaf-disease-detection, linked to GitHub
Status	Online; active deployment marked “Deployment successful”
Public URL	https://paddyguard-leaf-disease-detection-production.up.railway.app
Public networking	Domain routed to port 8002
Region and replicas	US East, 1 replica
Private networking	Enabled (IPv4 and IPv6); reachable inside Railway as paddyguard-leaf-disease-detectio
Outbound IPv6	Disabled
Deployment history	Earlier deployments of the same commit listed as REMOVED (superseded builds)
Account	Trial credit (24 days or $3.65 remaining when captured)

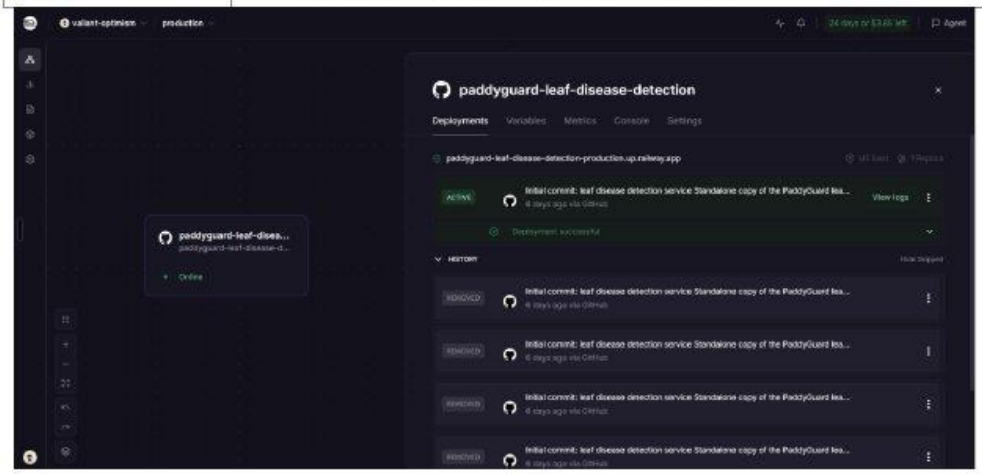


Figure 6.1: Railway Deployments tab with the service online and the active deployment successful

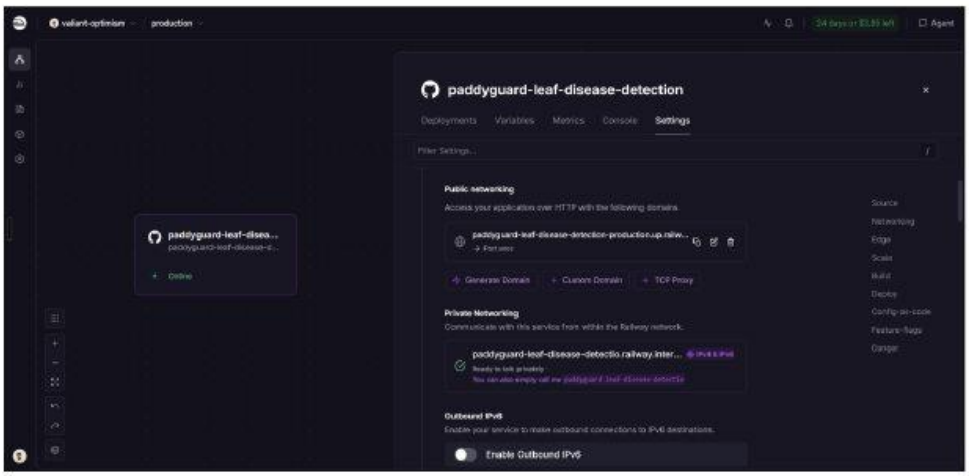


Figure 6.2: Railway Settings → Networking showing the public domain and port 8002

6.2 Containerisation

The image is based on `python:3.11-slim` and installs CPU-only PyTorch wheels, avoiding about 1.3 GB of unused CUDA libraries. Gunicorn runs one worker. Because Grad-CAM stores activations and gradients on the shared model object, the Railway repository serialises predictions with a lock so that the service can safely use several request threads. The trained checkpoint is copied into the image as a seed; an entrypoint script copies it into `/app/models` at start-up. That directory can be backed by a Railway persistent volume so that promoted models, backups and candidates survive redeployments, and on start-up the service loads the promoted model recorded in the active deployment record if present.

6.3 Environment Configuration

Variable	Value	Effect
MODEL_PATH	models/PaddyGuard_active_learning_round2.pth	Initial deployed checkpoint
OOD_ENERGY_THRESHOLD	3.0 (code default 5.0)	Energy below this is OOD
UNCERTAIN_THRESHOLD	0.70	Confidence below this is UNCERTAIN
LOW_CONFIDENCE_THRESHOLD	0.50	Case queued for expert review
MIN_FINE_TUNE_SAMPLES	3	Minimum verified samples
FINE_TUNE_EPOCHS	3	Two head-only plus one full epoch
TEST_DATASET_PATH	data/test_dataset	Held-out set used by the metric gate
MAX_BACKUPS / MAX_REJECTED_CANDIDATES	2 / 3	Retention limits

Variable	Value	Effect
TRAINING_LOG_RETENTION_DAYS	14	Log clean-up
AUTH_TOKEN_MAX_AGE_SECONDS	28800	Token lifetime (8 hours)
Database, Cloudinary, administrator and token secrets	Set in Railway Variables	Never committed to the repository

The code default for the energy threshold (5.0) differs from the deployed value (3.0), so the Railway variable must be set explicitly; otherwise the service would silently use the stricter gate that rejects 10.7% of genuine leaves.

6.4 Integration with the PaddyGuard Gateway

The gateway reads the leaf service address from `LEAF_DISEASE_URL`. Setting it to the Railway URL connected the platform to the cloud component with no gateway code change; the same code also works with a local container or Docker Compose. Integration issues resolved with the team were hard-coded ports clashing in Docker Compose, credentials hard-coded in an early frontend version (moved to environment variables), and routing of multipart uploads through the gateway.

6.5 Operation and Maintenance

- **Health monitoring:** `/health` reports whether the service is running, whether MongoDB is reachable and whether the model is loaded; Railway uses it for deployment checks.
- **Logging:** errors are logged with stack traces; noisy dashboard polling is filtered from the access log; each fine-tuning job writes its own log file.
- **Model operations:** the admin dashboard shows the active model, its accuracy, macro F1 and deployment date; changes are recorded and can be reversed from backups.
- **Housekeeping:** a retention routine runs at start-up and after each promotion.
- **Configuration:** thresholds and limits are environment variables and can be tuned without code changes.

6.6 Deployment Verification Checklist

- Confirm the deployment uses the dedicated GitHub repository and the correct root directory.
- Confirm Gunicorn binds to the port Railway routes to (8002 in the Networking tab, or the injected PORT).
- Confirm the checkpoint exists in the container and `/health` reports the model as loaded.
- Confirm `OOD_ENERGY_THRESHOLD=3.0` and the other variables are set in Railway Variables.
- Confirm whether a persistent volume is attached to `/app/models`, then redeploy and check that a promoted model survives.
- Submit a rice leaf and a non-rice image through the gateway and verify KNOWN and OOD responses.
- Record logs, the `/health` response and prediction screenshots as evidence (Appendix A).

6.7 Persistence and Operational Considerations

Without a volume, a container filesystem is ephemeral, so promoted models and backups would be lost on redeploy and the service would restart from the seed model. Cases, users, jobs and

deployment records are safe because they live in MongoDB Atlas, and images live in Cloudinary. The service runs from US East, which adds latency for users in Sri Lanka, and it currently runs on Railway trial credit, so a paid plan and a region closer to South Asia are needed for production use.

7. Testing and Results

7.1 Test Strategy

Testing followed ISO/IEC/IEEE 29119 levels: model evaluation, unit tests, API tests and end-to-end functional tests. Five test scripts cover active-learning eligibility, model retention, Grad-CAM analysis, authentication with duplicate upload, and an end-to-end uncertainty path with a mocked predictor. A Postman collection of 37 requests in 11 folders exercises every endpoint including error cases. A dedicated evaluation script reproduces the service's preprocessing and decision logic so that reported numbers reflect the deployed code path.

7.2 Selected Functional and Non-Functional Results

ID	Scenario	Result
TC01	Clear diseased leaf with field details	Pass
TC02	Non-rice image (dog)	Pass: rejected by the energy gate although softmax confidence was 72.27%
TC05	Confidence below 0.50 (mocked)	Partly met: backend verified, but the leaf page shows only KNOWN and OOD, so no UNCERTAIN card is displayed
TC06	Grad-CAM for a KNOWN result	Pass
TC09	Expert verifies a Top-K case	Pass (47 verified cases)
TC12	Regressing fine-tuning candidate	Pass: rejected, deployed model unchanged
TC13	Better candidate	Pass: promoted with backup, hot reload and deployment record
TC14	Failed fine-tuning job	Pass: marked FAILED, service kept serving
TC19	Duplicate upload within five minutes	Pass (code inspection and scripted test)
TC21	CPU inference time over 50 images	Pass: 42.0 ms; 254.8 ms with Grad-CAM (development CPU)
TC22	Accuracy on 838 test images	Pass: 97.14% (Round 2), 97.26% (promoted)
TC23	OOD rejection of at least 85%	**Not met:** 52.6% at the deployed threshold
TC24	Build and deploy from GitHub	Pass: service online, health check passed

7.3 Classification Results

Model	Training labels	Accuracy	Macro precision	Macro recall	Macro F1

Model	Training labels	Accuracy	Macro precision	Macro recall	Macro F1
Initial (full data)	4,200	97.85%	0.9780	0.9791	0.9784
Round 2 (deployed)	2,300	97.14%	0.9710	0.9724	0.9714
Promoted after fine-tuning	2,300 + 4 verified	97.26%	0.9726	0.9737	0.9728

Brown Spot is the hardest class (recall 94.67% for the initial model), mostly confused with Leaf Blast. All 194 Healthy test images were classified correctly by the Round 2 model.

7.4 OOD Detection Results

At the deployed energy threshold of 3.0 with the three-state logic:

Input set	KNOWN	UNCERTAIN	OOD
Rice leaf test set (838)	826	6	6
OOD test set (194)	86	6	102

Only 0.72% of genuine rice leaves were wrongly rejected and 98.06% of the 826 diagnosed leaves were correct. However, 52.6% of OOD images were rejected and 44.3% still received a diagnosis. All 45 weed images and 28 of 37 soil images were rejected, but 49 of 60 banana leaves and 29 of 52 mango leaves were diagnosed as rice diseases. AUROC was 0.9563 (energy) and 0.9664 (MSP) for the deployed model. A threshold of 5.0 rejects 88.1% of OOD images but wrongly rejects 10.7% of genuine leaves, which is why 3.0 was deployed.

7.5 Gated Fine-Tuning Results

Of twelve fine-tuning jobs, ten produced candidates 0.12–0.72 percentage points worse than the deployed model and were rejected, one (job 9, 97.26%) was promoted, and one failed without affecting the service. With only three to eight new samples per job the pipeline is valuable for its safety, not for measurable accuracy gains.

7.6 Operational Results

The model has 10.7 million parameters and a 41.4 MB checkpoint. Classification took 42.0 ms on average (95th percentile 56.5 ms) and 254.8 ms with Grad-CAM (95th percentile 308.4 ms) on a development CPU with 14 threads. Latency on the Railway instance has not been measured separately and is expected to be higher.

8. Challenges and Resolutions

Challenge	Impact	Resolution
Near-OOD images (banana, mango) resemble rice leaves	Only 52.6% OOD rejection at the deployed threshold	Three-state logic, Grad-CAM and expert review; future work on a combined gate and larger OOD training data
Code default threshold (5.0) differs from deployed (3.0)	Wrong gate if the variable is missing	Set OOD_ENERGY_THRESHOLD in Railway Variables and verify after each deploy

Challenge	Impact	Resolution
Grad-CAM shares state on the model object	Unsafe with several threads	One Gunicorn worker plus a prediction lock in the Railway repository
Developers shared one database but kept separate model files	Fine-tuning jobs 11 and 12 measured a 97.14% baseline instead of 97.26%	Run all fine-tuning on the single deployed instance with the model directory on persistent storage
Hard-coded ports and credentials during integration	Port clashes and exposed secrets	Environment variables and Compose cleanup
Ephemeral container filesystem	Promoted models could be lost on redeploy	Persistent volume at /app/models and a persistence test
US East region and trial credit	Latency for Sri Lankan users; no continuous operation	Move to a closer region on a paid plan

9. Current Status and Remaining Work

Work item	Status
Classifier, OOD gate, Grad-CAM, three-state logic	Implemented and evaluated
Expert review, active learning, gated fine-tuning	Implemented and tested
Railway deployment and gateway integration	Deployed and online
OOD rejection target (85%)	Not met (52.6%)
UNCERTAIN result card in the leaf page	Not implemented
Hosted-instance latency measurement	Pending
Persistent volume and restart persistence test	To be confirmed
Field study with farmers and extension officers	Not carried out
Sri Lankan field images and systematic model comparison	Future work

Planned improvements are a combined or learned OOD gate trained with near-OOD examples, a UNCERTAIN card in the interface, unified authentication with the shared user service, a closer cloud region, and collection of local field images.

10. Conclusion

Component 2 is deployed as a containerised Flask service on Railway, connected to the PaddyGuard gateway, and delivers 97.14–97.26% classification accuracy with Grad-CAM explanations, an expert review workflow and a gate that protected the live model from ten regressing candidates. The main shortfall is OOD rejection of near-OOD images (52.6% against an 85% target), which the three-state design and expert review partly mitigate. Before final submission the Railway evidence listed in Appendix A should be completed.

References

[1] M. Tan and Q. V. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," ICML, 2019, arXiv:1905.11946.

[2] W. Liu, X. Wang, J. Owens and Y. Li, "Energy-based out-of-distribution detection," NeurIPS, 2020, arXiv:2010.03759.

[3] R. R. Selvaraju et al., "Grad-CAM: Visual explanations from deep networks via gradient-based localization," ICCV, 2017, arXiv:1610.02391.

[4] D. Hendrycks and K. Gimpel, "A baseline for detecting misclassified and out-of-distribution examples in neural networks," ICLR, 2017, arXiv:1610.02136.

[5] H. M. L. M. Hewanayake, "Explainable rice leaf disease classification with out-of-distribution detection and human-in-the-loop active learning," individual thesis, SLIIT, 2026.

Appendix A – Evidence to Attach

Evidence	Status
Railway Deployments tab (service online, active deployment successful)	Included (Figure 6.1)
Railway Settings → Networking (public domain, port 8002)	Included (Figure 6.2)
Live `/health` response showing database and model loaded	To add
Railway Variables tab with secret values hidden	To add
Deploy logs showing start-up and model load	To add
Successful analysis and OOD rejection through the gateway	To add
Volume settings for /app/models and restart persistence test	To add

Appendix B – Abbreviations

AUROC: Area Under the ROC Curve · API: Application Programming Interface · CNN: Convolutional Neural Network · CORS: Cross-Origin Resource Sharing · Grad-CAM: Gradient-weighted Class Activation Mapping · MSP: Maximum Softmax Probability · OOD: Out-of-Distribution · REST: Representational State Transfer · WSGI: Web Server Gateway Interface

4. Component 3 — Rice Pest Detection

Original report reproduced in full, without summarization or omission.

PADDYGUARD AI

A Multimodal AI-Based System for Rice Leaf Disease and Pest Detection, Treatment Recommendation and Knowledge Assistance

DEVELOPMENT & DEPLOYMENT REPORT

Component 4: Rice Pest Detection, Image Quality Assessment, OOD Detection and Explainable AI

Project ID	R26-SE-015
Student	E P K Kalhara
Student ID	IT22065858
Report Date	October 2026

Declaration

I declare that this report describes the work I carried out for the Rice Pest Detection component of PaddyGuard AI. The report covers the model selected for the component, image-quality checking, unknown-input detection, Grad-CAM visualisation, support for adding new pest classes, and deployment of the FastAPI backend. The results and screenshots included in the final version should match the experiments and deployment that I personally completed.

Student: E P K Kalhara
Student ID: IT22065858
Signature: _____
Date: _____

Supervisor acknowledgement :
Name: Dr. Nathali Silva
Signature: _____
Date: _____

Acknowledgement

I would like to thank my project supervisor for the guidance and feedback provided during this research. I also thank the PaddyGuard AI team members for their work on the other components of the system. Finally, I am grateful to my family and friends for their encouragement while I developed and tested my component.

Executive Summary

This report describes my work on the Rice Pest Detection component of PaddyGuard AI. The system analyses uploaded images to classify rice pests and includes additional functions for checking image quality, flagging potentially unfamiliar inputs, generating Grad-CAM visualisations, and registering a new pest class. The backend is written in Python using FastAPI, and the image classifier selected for the current implementation is DenseNet121.

The backend provides routes for health checks, image prediction, few-shot class registration, selective fine-tuning, listing learned classes, and deleting a learned class. During prediction, the image is checked for basic quality issues before classification. The service can also calculate a Mahalanobis-distance OOD score when the required reference data is available, and it can return a Grad-CAM visualisation for supported predictions.

I trained and compared five candidate models and selected DenseNet121, which achieved 93% accuracy in my evaluation. The final report should include the test-set details and the original evaluation output so that this result can be checked. The deployment also depends on the trained model checkpoint and, for Mahalanobis OOD detection, the reference-statistics file. Their availability in the live Railway service must be confirmed before submission.

The report follows the main sections used in the project team's development reports. Yellow-highlighted text marks details, results, or evidence that I need to check or add before submitting the final version.

Table of Contents

Table of Contents

Declaration....2
Acknowledgement....2
Executive Summary....2
Table of Contents....3
1. Introduction....5
1.1 Background and Context....5
1.2 Problem Statement and Research Gap....5
1.3 Scope of This Report....5
2. Objectives and Research Questions....5
2.1 Main Objective....5
2.2 Specific Objectives....5
2.3 Research Questions....6
3. Development Methodology....6
3.1 Research Approach....6
3.2 Development Workflow....6
3.3 Dataset and Image Preprocessing....7
3.4 Model Development and Selection....7
3.5 Research Integrity and Evidence Boundaries....7
4. System Architecture....8
4.1 Position within PaddyGuard AI....8
4.2 Backend Architecture....8
4.3 Prediction Data Flow....8
4.4 API Design....8
5. Component Implementation....9
5.1 DenseNet121 Classifier....9
5.2 Image Quality Assessment....9
5.3 Out-of-Distribution Detection....9
5.4 Grad-CAM Explainability....10
5.5 Few-Shot Learning and Selective Fine-Tuning....10
5.6 Error Handling and Response Design....10
6. Cloud Deployment and Integration....10
6.1 Railway Deployment Model....10
6.2 Containerization....11
6.3 Environment Configuration....11

6.4 Deployment Verification Checklist 11
6.5 Persistence and Operational Considerations 12
7. Testing and Current Results 12
7.1 Automated Tests 12
7.2 Functional Test Scenarios 12
7.3 Classification Result Reported by the Student 13
7.4 OOD and Image Quality Evaluation 13
8. Challenges and Resolutions 13
9. Current Status and Remaining Work 14
10. Evaluation Plan 15
11. Conclusion 15
References 16
Appendix A – API Endpoints 17
Appendix B – Environment Variables 17
Appendix C – Evidence Screenshots to Insert 18
Appendix D – Abbreviations 20

1. Introduction

1.1 Background and Context

Rice cultivation is important to food security and livelihoods in Sri Lanka. Pest infestations can damage rice plants, reduce yield, and increase the cost of cultivation. Timely identification helps farmers decide when to seek advice or apply appropriate management practices. However, identifying small insects from field photographs can be difficult because images vary in lighting, focus, resolution, distance, and background complexity.

Computer vision and deep learning provide a practical way to analyse crop images and assign them to known pest categories. A conventional closed-set classifier, however, generally returns one of its trained labels even when an image contains an unfamiliar insect or an irrelevant object. A reliable agricultural tool should therefore check input quality, communicate uncertainty, and avoid presenting every prediction as definitive.

PaddyGuard AI is a multimodal research platform that combines image-based disease and pest recognition with voice-based symptom diagnosis and treatment advisory functions. This report focuses only on the Rice Pest Detection component, including its image-quality checks, DenseNet121 inference, optional out-of-distribution detection, Grad-CAM explanation, and support for registering new pest classes.

1.2 Problem Statement and Research Gap

The practical problem addressed by this component is the need for an accessible image-based rice pest identification service that remains cautious when the submitted image is unsuitable or unfamiliar. The component addresses three connected issues: variable image quality, classification among known rice pest categories, and the risk of confidently assigning a known label to an unknown input.

The development extends beyond a basic classifier by adding a quality assessment stage, an optional Mahalanobis-distance detector, visual explanation, and mechanisms for learning additional categories from a small number of labelled examples. These features improve the engineering scope of the system, but their real-world reliability must be assessed through controlled experiments and field-image testing.

1.3 Scope of This Report

This report describes the development and deployment of the Rice Pest Detection component of PaddyGuard AI. It covers the dataset preparation, training and evaluation of five deep learning models, selection of DenseNet121 as the best-performing model, and the implementation of image quality assessment, Out-of-Distribution (OOD) detection, and Grad-CAM visualisation. It also discusses few-shot learning and fine-tuning for adding new pest classes, the FastAPI backend, and deployment on Railway. The report concludes with the system testing, evaluation results, limitations, and possible future improvements.

2. Objectives and Research Questions

2.1 Main Objective

To develop and deploy a deep learning-based rice pest detection service that classifies images into known pest categories, checks image suitability, flags unfamiliar inputs where the OOD reference is available, explains predictions visually, and supports the addition of new pest classes.

2.2 Specific Objectives

- Prepare and preprocess rice pest images for consistent model inference.

- Train and compare five candidate deep learning architectures and select the best-performing model based on evaluation results.
- Implement a DenseNet121-based classifier for five rice pest categories.
- Assess uploaded images for low resolution, blur, unsuitable brightness, low contrast, and excessive background edge density.
- Provide an OOD mechanism based on Mahalanobis distance when reference embeddings and covariance statistics are available.
- Generate Grad-CAM heatmaps to indicate image regions that influence a selected prediction.
- Support prototype-based few-shot registration and selective fine-tuning using a small set of labelled images.
- Expose the component through documented HTTP endpoints suitable for integration with the wider PaddyGuard AI frontend.

2.3 Research Questions

- How effectively can transfer-learning CNN architectures classify the selected rice pest categories?
- Does image-quality screening help identify images that are unsuitable for dependable inference?
- How effectively can the OOD mechanism distinguish unfamiliar inputs from the learned pest distribution?
- Can Grad-CAM provide useful visual evidence about the regions influencing model predictions?
- Can few-shot prototypes or selective fine-tuning provide a practical path for registering additional pest categories from a limited number of labelled images?
- Can the backend be packaged and deployed as a maintainable HTTP service on Railway?

3. Development Methodology

3.1 Research Approach

The work follows an applied design-and-evaluate approach. The engineering artefact is a REST API that wraps an image classifier and supporting reliability functions. Development is incremental: model loading, preprocessing, image-quality assessment, prediction, OOD checks, visual explanation, new-class registration, and API handling are separated into modules so that they can be tested and maintained independently.

3.2 Development Workflow

Stage	Activity	Evidence or output
1. Requirements	Define target pest classes, image input contract, and reliability features.	Project proposal and backend API schemas.
2. Data preparation	Prepare and organize the rice pest image dataset, apply image preprocessing and normalization, and divide the data into training, validation, and testing sets.	Prepared dataset, image preprocessing code, and dataset split details.
3. Model experiments	Train and evaluate five candidate deep learning models using the prepared dataset and select DenseNet121 based on the evaluation results.	Model evaluation results showing DenseNet121 as the selected model, with an accuracy of 93%.
4. Inference implementation	Load DenseNet121, preprocess images, compute class probabilities, and return the predicted label.	`pipeline/ml/model_loader.py`.
5. Reliability layer	Add quality assessment, optional Mahalanobis OOD, Grad-CAM, few-shot prototypes, and fine-tuning.	`pipeline/services/` and `pipeline/ml/`.
6. API integration	Expose prediction and learning endpoints with input validation and structured responses.	`api/endpoints.py` and `schemas/`.

7. Deployment	Deploy the FastAPI backend to Railway using the project configuration and ensure that the service starts correctly on the assigned port.	main.py, Railway deployment configuration, and screenshots of the successful deployment.
8. Verification	Run automated tests and verify live health/prediction routes and edge cases.	Test files are included; test-run logs and live HTTP evidence are not in the archive.

3.3 Dataset and Image Preprocessing

The Rice Pest Detection component uses five target classes: Brown Planthopper, Rice Gall Midge, Rice Hispa, Rice Leaf Folder, and Rice Stem Borer. The image dataset is used to train and evaluate the candidate deep learning models, with DenseNet121 selected as the best-performing model based on the experimental results. The dataset is organized to support pest classification and the evaluation of model performance.

During inference, uploaded images are decoded using Pillow and converted to RGB format. The images are resized to 224 × 224 pixels and converted into PyTorch tensors for DenseNet121 prediction. Image preprocessing is applied consistently before inference to prepare the input for the model. The image quality assessment service also checks image dimensions, blur, brightness, contrast, and edge density to identify potentially unsuitable inputs.

The quality service computes image width and height, Laplacian variance as a blur indicator, grayscale mean as brightness, grayscale standard deviation as contrast, and Canny edge density as a rough measure of background complexity. Configurable thresholds are used to produce warnings. The quality gate is heuristic: it does not prove that a pest is visible or that the image is biologically valid.

3.4 Model Development and Selection

The backend loads a DenseNet121 architecture from Torchvision and replaces its classification head with a dropout layer followed by a linear layer for five classes. The checkpoint is loaded from the configured `MODEL_PATH` (default: `models/best_model.pth`) with strict state-dictionary matching. The model is switched to evaluation mode and uses CUDA when available, otherwise CPU.

Five deep learning models were evaluated during the model selection process: ResNet50, EfficientNetB0, MobileNetV2, DenseNet121, and InceptionV3. Based on the experimental results, DenseNet121 was selected as the final model for the Rice Pest Detection component, achieving an accuracy of 93% on the evaluation dataset.

Item	Current documented value	Evidence status
Selected classifier	DenseNet121	Confirmed by source code.
Number of models compared	Five	Model comparison results
Accuracy	93%	Model comparison results
Input size	224 × 224 pixels	Source-code configuration.
Base classes	Five rice pest categories	Source-code class mapping.
Runtime device	CUDA if available; otherwise CPU	Source-code logic.

3.5 Research Integrity and Evidence Boundaries

In this report, I describe the features implemented in the backend separately from the results measured during testing. The 93% accuracy figure comes from my model evaluation, while the final version should include the evaluation output and live Railway screenshots to document the model result and deployment.

4. System Architecture

4.1 Position within PaddyGuard AI

Within the wider PaddyGuard AI platform, the pest service accepts an image and returns a pest label, confidence value, status, quality information, optional OOD score, and Grad-CAM image data where available. The frontend can use this response to show the user a prediction or request a clearer image. The pest component is separate from the voice diagnosis, leaf-disease diagnosis, and treatment-advisory components.

4.2 Backend Architecture

Layer / module	Responsibility	Implementation
HTTP API	Health, prediction, class registration, fine-tuning, listing and deletion routes.	FastAPI
Input validation	Multipart file handling, image type/size checks, decode validation.	FastAPI, Pillow
Image quality	Blur, resolution, brightness, contrast and edge-density checks.	OpenCV, NumPy, Pillow
Base classifier	DenseNet121 model construction, checkpoint loading, class probabilities and feature embeddings.	PyTorch, Torchvision
OOD detector	Mahalanobis distance over class means and inverse covariance when reference statistics are available.	PyTorch
Explainability	Grad-CAM activation/gradient capture and heatmap overlay.	PyTorch, OpenCV
New-class support	Prototype similarity classification and a separately stored selective fine-tuning adapter.	PyTorch
Configuration	Environment-based settings and CORS allow-list.	pydantic-settings, FastAPI middleware
Deployment	Containerized Python API listening on configured service port.	Docker, Uvicorn, Railway target

4.3 Prediction Data Flow

Incoming image → HTTP upload validation → image decode to RGB → image-quality assessment → DenseNet121 prediction → optional Mahalanobis OOD score → if confidence is low or OOD is flagged, check fine-tuned adapters and then few-shot prototypes → determine final status and prediction → generate Grad-CAM when supported → return structured JSON response.

The prediction pipeline includes a conditional Out-of-Distribution (OOD) detection mechanism based on Mahalanobis distance. This mechanism operates when the required reference file is available. If the reference file cannot be loaded, the system uses a confidence-threshold fallback for low-quality inputs instead of producing a calibrated Mahalanobis score. Therefore, the availability of the required reference data affects the OOD functionality of the deployed system.

4.4 API Design

Endpoint	Method	Purpose
/	GET	Root status message and links to docs/health.
/health	GET	Root health status, model loading and device information.
/api/v1/health	GET	Component health response.

/api/v1/predict	POST	Accept one image in multipart field `file` and return prediction/reliability information.
/detect	POST	Compatibility route exposing the same prediction function.
/api/v1/few-shot/classes	GET	List registered prototype and fine-tuned classes.
/api/v1/few-shot/register	POST	Register a class from 5–20 labelled images using feature prototypes.
/api/v1/few-shot/fine-tune	POST	Adapt a separate DenseNet121 model for a new class using 5–20 labelled images.
/api/v1/few-shot/classes/{class_name}	DELETE	Remove a registered prototype and/or fine-tuned class.

5. Component Implementation

5.1 DenseNet121 Classifier

The classifier constructs a Torchvision DenseNet121 model without downloading pretrained weights at runtime (`weights=None`) and restores the trained state dictionary from the configured checkpoint. The classifier head is configured as Dropout(p=0.5) followed by a linear layer with five outputs. Softmax converts logits into class probabilities, and the maximum probability supplies the predicted class and confidence.

The class order encoded in the source is: Brown Planthopper, Rice Gall Midge, Rice Hispa, Rice Leaf Folder, and Rice Stem Borer. Training labels and checkpoint output order must match this exact order. A mismatch would result in labels being assigned to the wrong output indices.

5.2 Image Quality Assessment

Signal	Implementation	Purpose / caveat
Resolution	Checks width and height against minimum 224 pixels.	Rejects small inputs; does not establish focus or pest visibility.
Blur	Variance of the Laplacian of a grayscale image.	Low values trigger a blur warning; thresholds may vary with image content.
Brightness	Mean grayscale intensity.	Flags images below 35 or above 225 by default.
Contrast	Standard deviation of grayscale intensity.	Flags contrast below 18 by default.
Background complexity	Canny edge density.	Flags edge density above 0.38; busy natural scenes may trigger warnings.

If any quality warning is produced, the service returns an `Image Quality Too Low` result with confidence 0, status `unknown`, source `quality_check`, the warnings and quality measurements, and no Grad-CAM image. These thresholds are configuration defaults and should be tuned against a labelled set of acceptable and unacceptable images.

5.3 Out-of-Distribution Detection

The Mahalanobis OOD module compares a feature embedding with class-wise means using an inverse covariance matrix estimated from training embeddings. It computes the minimum squared Mahalanobis distance across the stored class means and flags an input as unknown when the score exceeds a configured threshold. The default threshold is 15.0, but it can be overridden by the reference payload.

The OOD detection mechanism is implemented using Mahalanobis distance and requires reference statistics generated from training data. The reference file, data/ood/reference.pt, was not included in the supplied ZIP archive,

which contained only a .gitkeep placeholder in the OOD directory. Therefore, although the OOD detection logic is present in the source code, its activation in the Railway deployment could not be verified. The reference statistics must be generated using the provided build script and made available to the deployed service. The confidence-threshold fallback does not provide the same functionality as calibrated Mahalanobis-based OOD detection.

5.4 Grad-CAM Explainability

The Grad-CAM module registers hooks on the final convolution layer `features.denseblock4.denselayer16.conv2` in DenseNet121. It records forward activations and backward gradients, computes class-specific weights, forms a heatmap, and overlays the heatmap on the source image. The response schema supports a base64-encoded Grad-CAM image.

Grad-CAM is an explanatory visualization, not a proof that the model uses biologically valid evidence. A research evaluation should inspect heatmaps across correct and incorrect predictions, multiple classes, and field backgrounds, and should report representative examples without cherry-picking only successful cases.

5.5 Few-Shot Learning and Selective Fine-Tuning

The API supports two paths for adding a new pest category. Prototype few-shot registration accepts 5–20 labelled images, extracts DenseNet121 feature embeddings, normalizes them, averages them into a class prototype, and saves the prototype as a `.pt` file. During prediction, cosine-like similarity is calculated using the dot product of normalized embeddings; a match is returned when the similarity meets the configured threshold (default 0.72).

Selective fine-tuning also accepts 5–20 labelled images. It clones the base DenseNet121, expands the classifier output for the new class, freezes most of the network, and fine-tunes selected later layers and the classifier. The adapted model is saved separately so that the original base checkpoint is not overwritten. The configured defaults include eight epochs, learning rate 1e-4, and a confidence threshold of 0.70. These are implementation defaults and should not be described as experimentally optimal without a controlled study.

The prediction service attempts these new-class paths when the base confidence is below the configured `maybe_threshold` (default 0.70) or OOD is flagged. It checks the fine-tuned adapter first, then the prototype store if no fine-tuned match is returned. The API also supports listing and deleting learned classes.

5.6 Error Handling and Response Design

The prediction route accepts JPEG, PNG, and WEBP MIME types, rejects empty or corrupted uploads, and enforces a configurable maximum file size (10 MB by default). Unsupported types return HTTP 415, oversized files return HTTP 413, malformed image data returns HTTP 400, and model runtime errors are logged and returned as HTTP 500.

The service uses Pydantic response schemas and FastAPI's generated OpenAPI documentation. The main route is `/api/v1/predict`; `/detect` is included as a compatibility route for research integrations. CORS origins are controlled through the `ALLOWED_ORIGINS` setting, with local development origins as the defaults.

6. Cloud Deployment and Integration

6.1 Railway Deployment Model

The Rice Pest Detection backend was deployed on Railway as a FastAPI application. The project includes a Dockerfile, a Uvicorn startup configuration, health-check endpoints, and environment-based settings. Railway provides the hosting environment for the backend API, allowing the application to be accessed through a public URL. The deployment is intended to support image-based pest prediction and the component's related backend functions.

6.2 Containerization

The Dockerfile uses `python:3.11-slim`, installs the system packages `libgl1` and `libglib2.0-0`, installs CPU-specific PyTorch 2.6.0 and Torchvision 0.21.0 from the PyTorch CPU index, installs the remaining Python requirements, copies the source tree, exposes port 8003, and starts Uvicorn with `--host 0.0.0.0 --port 8003`.

The application is containerized using Docker. The Dockerfile uses Python 3.11 Slim, installs the required system and Python dependencies, copies the application source code, and starts the FastAPI service using Uvicorn. The container is configured to expose port 8003. The application configuration and Railway port settings must be consistent to ensure that the service starts and handles requests correctly in the deployment environment.

6.3 Environment Configuration

Setting	Default in source	Deployment note
MODEL_PATH	models/best_model.pth	The trained checkpoint is required. It is not present in the supplied ZIP inventory.
IMAGE_SIZE	224	Must match training input size.
MAX_UPLOAD_SIZE_MB	10	Limits each uploaded image.
UNKNOWN_THRESHOLD	0.50	Declared configuration; inspect service usage before claiming it governs all OOD decisions.
MAYBE_THRESHOLD	0.70	Triggers new-class matching when base confidence is low.
MIN_WIDTH / MIN_HEIGHT	224 / 224	Minimum dimensions for quality gate.
BLUR_THRESHOLD	35.0	Laplacian-variance cutoff.
MIN_BRIGHTNESS / MAX_BRIGHTNESS	35 / 225	Grayscale brightness cutoffs.
MIN_CONTRAST	18.0	Grayscale standard-deviation cutoff.
MAX_EDGE_DENSITY	0.38	Background complexity heuristic.
ALLOWED_ORIGINS	localhost:3000 and localhost:5173	Set the production frontend origin for browser requests.
OOD_REFERENCE_PATH	data/ood/reference.pt	Required for Mahalanobis OOD to be ready.
OOD_MAHALANOBIS_THRESHOLD	15.0	Threshold can be overridden by the reference payload.
FINE_TUNE_EPOCHS / LEARNING_RATE	8 / 1e-4	Fine-tuning defaults.
FINE_TUNE_CONFIDENCE_THRESHOLD	0.70	Confidence cutoff for fine-tuned adapter matching.

6.4 Deployment Verification Checklist

- Confirm the Railway deployment uses the intended repository or uploaded source archive and the correct root directory.
- Confirm the start command and service port match Railway's runtime configuration.
- Verify that `models/best_model.pth` is available in the deployed container and that the model loads successfully.
- Verify that `data/ood/reference.pt` is available when Mahalanobis-based OOD detection is enabled.
- Set `ALLOWED_ORIGINS` to the exact production frontend origin; do not use a wildcard with credentials.
- Open `/` and `/docs` on the public service URL and verify the root message and OpenAPI page.

- Access the deployed health endpoint and verify the service status, model loading status, and runtime device reported by the API.
- Submit a valid rice pest image to `/api/v1/predict` and examine the predicted class, confidence score, image quality result, OOD information, and Grad-CAM output where supported.
- Test invalid MIME type, empty file, corrupted image, oversized file, and low-quality image handling.
- If new-class registration is enabled, register a temporary pest class using labelled images, test the prediction behaviour, and remove the class after testing if appropriate.
- Redeploy or restart the service and check whether learned prototypes/adapters persist.
- Record the actual Railway deployment status, relevant logs, health endpoint response, API documentation page, and successful prediction results as evidence of deployment and testing.

6.5 Persistence and Operational Considerations

The source saves few-shot prototypes and fine-tuned adapters as files under `data/few_shot` and `data/fine_tuned`. In a container platform, the durability of these files depends on the filesystem and volume configuration. If the service uses an ephemeral filesystem, new classes may disappear after a redeploy or instance replacement. Persistent Railway volumes or external object storage should be considered if learned classes must survive restarts.

The deployed service requires the trained model checkpoint and the OOD reference file for their respective functions. These files must be available to the Railway application at runtime. The availability of the model checkpoint and the OOD reference file should be verified in the deployed environment. If the OOD reference file is unavailable, Mahalanobis-based OOD detection cannot be confirmed as active in production and should be reported as an implemented feature requiring deployment verification.

7. Testing and Current Results

7.1 Automated Tests

The project includes automated tests for the detector, API endpoints, and image-quality checks. I should run these tests in the current development environment and include the actual pass/fail output. The presence of test files alone is not evidence that every test has passed.

Test file	Intended coverage	Evidence supplied
tests/test_detector.py	Detector module imports successfully.	Test source included; no execution log.
tests/test_endpoints.py	Root route and health route are registered.	Test source included; no execution log.
tests/test_quality.py	Quality service accepts a synthetic clear image.	Test source included; no execution log.
tests/conftest.py	Shared test fixtures/configuration.	Test source included.

7.2 Functional Test Scenarios

Test ID	Input / action	Expected behaviour	Status in this report
T01	GET `/`	Return service-running message and links.	Route exists in source; live test evidence pending.
T02	GET `/api/v1/health`	Return status, model-loaded state, device and model name.	Route exists in source; checkpoint availability must be confirmed.
T03	POST valid JPEG/PNG/WEBP	Return prediction, confidence, quality and explanation fields.	Implemented in source; live response screenshot pending.
T04	Unsupported MIME type	Return HTTP 415.	Validation implemented in source.

T05	Oversized image	Return HTTP 413.	Validation implemented in source.
T06	Corrupt or empty image	Return HTTP 400.	Validation implemented in source.
T07	Image below quality thresholds	Return quality warning and avoid normal classification.	Implemented in prediction service.
T08	Unknown image with OOD reference	Use Mahalanobis score and return unknown status if threshold exceeded.	Conditional; reference asset missing from supplied ZIP.
T09	Register 5–20 new-class images	Save prototype or adapted model and return registration result.	Implemented in source; persistence and live testing evidence pending.
T10	Restart/redeploy after registration	Verify learned class availability persists.	Requires a deployment persistence test.

7.3 Classification Result Reported by the Student

Five candidate deep learning models were evaluated for the Rice Pest Detection component. Based on the experimental results, DenseNet121 was selected as the best-performing model, achieving 93% classification accuracy. The model was selected for backend integration based on its performance during evaluation.

Metric	Value	Current evidence
Best model	DenseNet121	Source code and student statement.
Number of trained models	5	Student statement; model comparison artefact not supplied.
Accuracy	93%	Reported evaluation result
Precision / Recall / F1	Not supplied	Must be filled from actual evaluation output.
Test-set size and class distribution	Not supplied	Must be filled from dataset manifest or evaluation log.
Mean inference latency	Not supplied	Evaluation script can calculate it; actual run output required.

7.4 OOD and Image Quality Evaluation

The Rice Pest Detection component includes image-quality assessment and Mahalanobis-based Out-of-Distribution (OOD) detection. However, the supplied source archive does not contain the required OOD reference file or completed OOD evaluation results. Therefore, OOD performance cannot yet be quantified using metrics such as AUROC, rejection rate, or false-positive rate. Further testing is required using known rice pest images, unfamiliar insects, non-insect images, and unrelated photographs, with the detection threshold selected using a validation dataset.

8. Challenges and Resolutions

Challenge / risk	Impact	Resolution or action
Model checkpoint absent from supplied ZIP	The model loader raises FileNotFoundError if `best_model.pth` is missing.	Verify how the checkpoint is included in Railway runtime and document the mechanism.
OOD reference file absent from supplied ZIP	Mahalanobis detector is not ready without `reference.pt`.	Generate reference statistics using the provided script and provision the file at runtime.

Port configuration mismatch risk	Dockerfile starts Uvicorn on 8003 while Railway may inject a `PORT`.	Use the platform-provided port in the container command or confirm fixed-port configuration.
Training preprocessing not fully documented	Inference transform resizes and converts to tensor without explicit normalization.	Verify and document exact training transforms and reproduce them in inference.
Quality thresholds may be data-dependent	Heuristics can reject valid images or pass unsuitable images.	Calibrate thresholds on labelled images captured in realistic field conditions.
OOD threshold not empirically calibrated	A default threshold may not generalize to unseen pests and backgrounds.	Use validation OOD sets and report AUROC, FPR95, precision/recall, and threshold rationale.
Learned classes stored on local filesystem	Classes may be lost after redeployment if storage is ephemeral.	Use a persistent volume or external storage and test recovery after restart.
Accuracy result lacks attached evidence	The reported 93% accuracy requires supporting experimental results for independent verification.	Include the available model comparison results, evaluation logs, test-set details, and classification metrics.
Live Railway evidence absent from archive	Public availability and actual runtime assets cannot be independently verified from source alone.	Insert Railway dashboard, deployment logs, health response, and successful prediction screenshots.

9. Current Status and Remaining Work

Work item	Status based on supplied material	Notes
FastAPI service and routing	Implemented in source	`main.py`, `api/endpoints.py`.
DenseNet121 inference	Implemented in source	Requires checkpoint file at configured path.
Five base pest classes	Implemented in source	Exact class names encoded in `model_loader.py`.
Image-quality assessment	Implemented in source	Threshold-based checks in `image_quality.py`.
Mahalanobis OOD	Implemented conditionally	Requires `data/ood/reference.pt`; file absent from supplied ZIP.
Grad-CAM	Implemented in source	Hook-based implementation and base64 response support.
Prototype few-shot registration	Implemented in source	Accepts 5–20 images; persistence needs deployment verification.
Selective fine-tuning	Implemented in source	Saves a separate adapter; performance evaluation still needed.
Five-model comparison and 93% accuracy	Reported experimental result	Include the model comparison table and evaluation results to support the reported accuracy.
Railway deployment	Reported by student	Attach public URL and dashboard/HTTP screenshots.
Automated tests	Test files included	Attach `pytest` output from a clean environment.
Field evaluation and user study	Not evidenced in supplied archive	Recommended future work.

10. Evaluation Plan

The following plan provides a reproducible way to evaluate the component after confirming that the correct checkpoint and reference assets are available in the deployed environment.

Experiment	Method	Metrics / evidence
Closed-set classification	Evaluate on a held-out, class-stratified test set not used during training or threshold selection.	Accuracy, per-class precision/recall/F1, macro F1, confusion matrix, class support.
Model comparison	Run the same test split and preprocessing for all five candidate models.	Comparison table with identical metrics and model inference time.
Image quality gate	Label images as acceptable/unacceptable with a documented criterion and compare the automatic gate.	Sensitivity to poor images, false-rejection rate, warning frequency.
OOD detection	Use held-out known-class images and separate near-OOD/unrelated image sets.	AUROC, AUPR, FPR at 95% TPR, rejection precision/recall, threshold.
Grad-CAM review	Inspect representative correct/incorrect predictions across classes and field backgrounds.	Qualitative examples and, if feasible, a structured human relevance rating.
Few-shot learning	Register new classes with 5, 10, and 20 labelled images; evaluate on separate images.	Accuracy/macro F1 per support size; similarity threshold sensitivity.
Fine-tuning	Compare base model and adapted model on held-out examples of new and original classes.	New-class recall, original-class retention, confusion matrix, training time.
Deployment performance	Send repeated requests to the live service and include cold starts and restarts.	HTTP status, latency percentiles, memory use if available, failure rate.
Persistence	Register a class, restart/redeploy, and check whether it remains available.	Before/after class list and storage configuration evidence.

11. Conclusion

The Rice Pest Detection component is implemented as a FastAPI service using DenseNet121 for five rice pest classes. It also includes image-quality checks, a Mahalanobis-distance OOD module, Grad-CAM visualisation, prototype-based few-shot registration, and selective fine-tuning for adding a new class. The API provides prediction and health routes as well as endpoints for managing learned classes.

My model comparison identified DenseNet121 as the best-performing candidate, with 93% accuracy in the evaluation I carried out. The final submission should include the original evaluation evidence and confirm that the model checkpoint and OOD reference data are available in the Railway runtime. The live service should also be tested after deployment so that the report reflects the actual running version.

Before final submission, I will verify the Railway deployment, capture screenshots of the health and prediction endpoints, attach the model comparison results, and check the OOD behaviour using suitable known and unfamiliar images. I will also test the image-quality checks and the process for registering a new pest class.

References

[1] K. P. Ferentinos, “Deep learning models for plant disease detection and diagnosis,” Computers and Electronics in Agriculture, vol. 145, pp. 311–318, 2018, doi: 10.1016/j.compag.2018.01.009.

[2] S. Sladojevic, M. Arsenovic, A. Anderla, D. Culibrk, and D. Stefanovic, “Deep neural networks based recognition of plant diseases by leaf image classification,” Computational Intelligence and Neuroscience, vol. 2016, Article ID 3289801, 2016, doi: 10.1155/2016/3289801.

[3] Y. Zheng, X. Wang, and H. Li, “Lightweight rice pest detection using improved YOLOv8 architecture,” Scientific Reports, vol. 14, 2024, doi: 10.1038/s41598-024-81587-5.

[4] J. Deng, Z. Chen, and Y. Zhang, “Deep learning-based crop disease and insect pest detection for mobile agricultural applications,” Agronomy, vol. 13, no. 8, 2023, doi: 10.3390/agronomy13082139.

[5] K. Hu, J. Zhang, and Y. Liu, “Rice pest recognition using deep convolutional neural networks,” Frontiers in Plant Science, vol. 14, 2023, doi: 10.3389/fpls.2023.1167121.

[6] W. Wang, Y. Chen, and Z. Li, “Deep learning approaches for plant disease and pest detection: A comprehensive review,” Remote Sensing, vol. 17, no. 4, 2025, doi: 10.3390/rs17040698.

[7] S. Liang, Y. Li, and R. Srikant, “Enhancing the reliability of out-of-distribution image detection in neural networks,” ICLR, 2018, doi: 10.48550/arXiv.1706.02690.

[8] Y.-C. Hsu, Y. Shen, H. Jin, and Z. Kira, “Generalized ODIN: Detecting out-of-distribution images without learning from out-of-distribution data,” CVPR, 2020, doi: 10.1109/CVPR42600.2020.01096.

[9] O. Ronneberger, P. Fischer, and T. Brox, “U-Net: Convolutional networks for biomedical image segmentation,” MICCAI, 2015, pp. 234–241, doi: 10.1007/978-3-319-24574-4_28.

[10] J. Devlin, M. W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” NAACL-HLT, 2019, pp. 4171–4186, doi: 10.18653/v1/N19-1423.

[11] D. P. Hughes and M. Salathé, “An open access repository of images on plant health to enable the development of mobile disease diagnostics,” arXiv, 2015, doi: 10.48550/arXiv.1511.08060.

Appendix A – API Endpoints

The exact API contract should be checked in the generated OpenAPI page (`/docs`) for the deployed version. The following table is derived from the supplied source.

Endpoint	Request	Expected use
GET /	No body	Service root and documentation links.
GET /health	No body	Root health check; loads the prediction service and reports model/device.
GET /api/v1/health	No body	Health response for the component API.
POST /api/v1/predict	Multipart form field `file`	Main image prediction.
POST /detect	Multipart form field `file`	Compatibility endpoint for research integration.
GET /api/v1/few-shot/classes	No body	List registered learned classes.
POST /api/v1/few-shot/register	Form field `class_name` + 5–20 image files	Prototype-based new-class registration.
POST /api/v1/few-shot/fine-tune	Form field `class_name` + 5–20 image files	Selective fine-tuning for a new class.
DELETE /api/v1/few-shot/classes/{class_name}	Class name path parameter	Remove stored learned class.

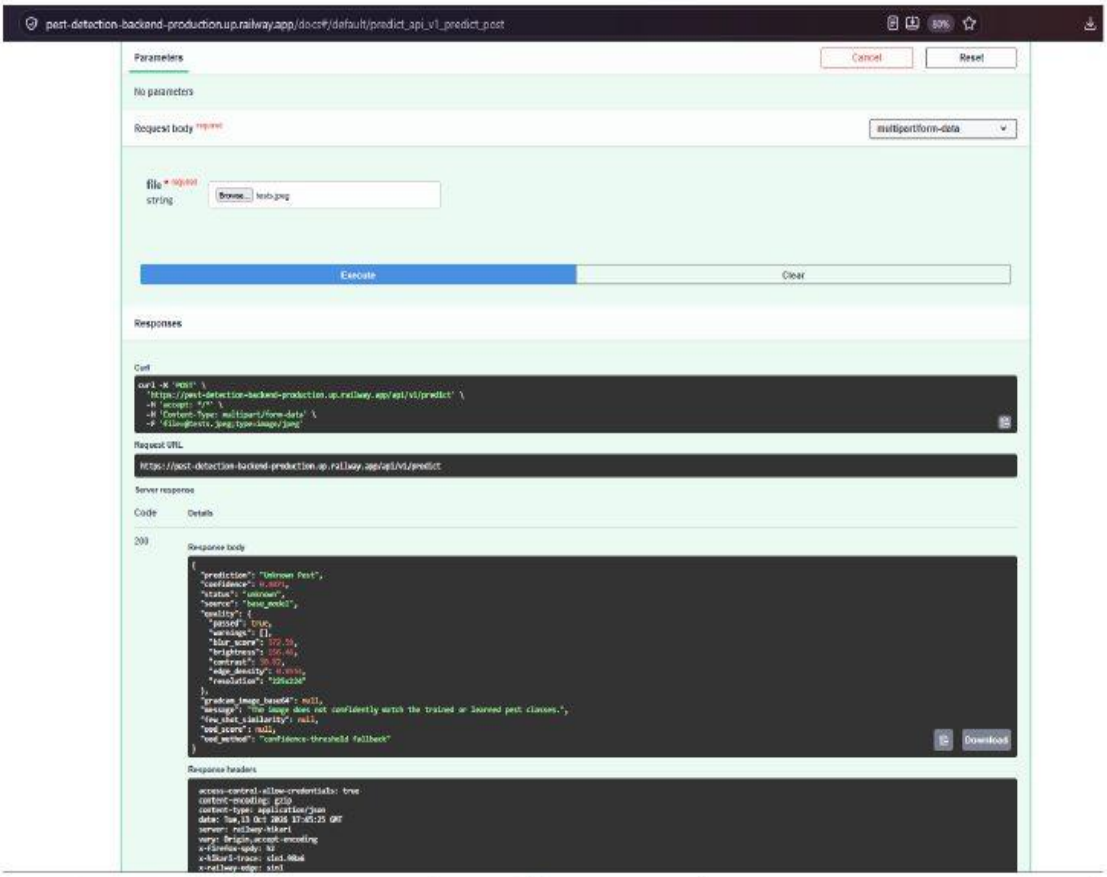
Appendix B – Environment Variables

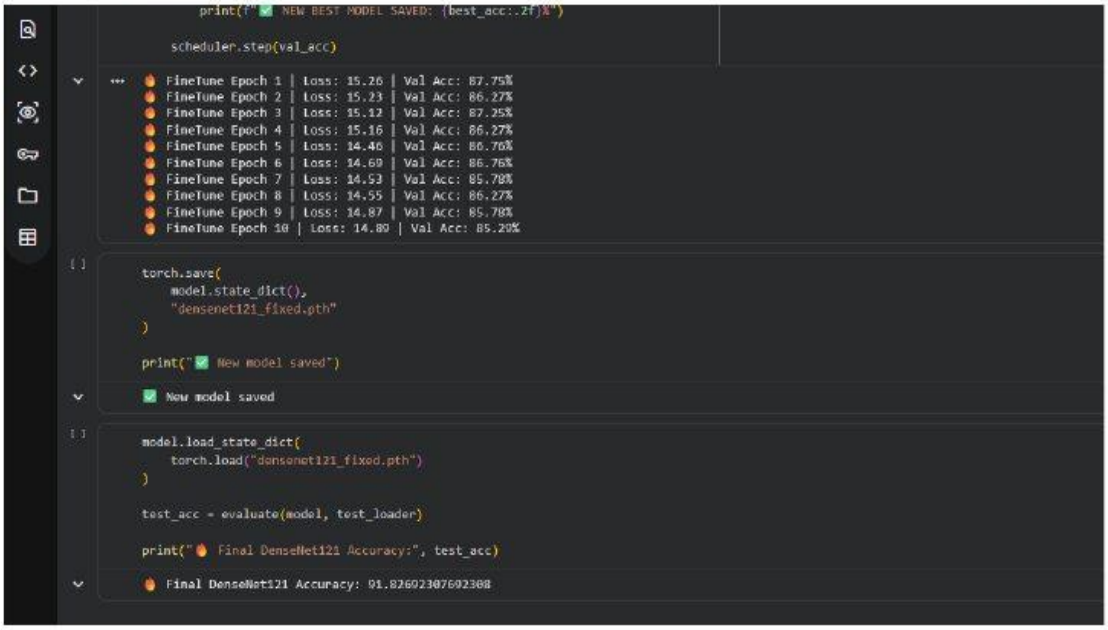
Configuration is loaded using `pydantic-settings` from environment variables and an optional `.env` file. Do not commit secrets or private deployment tokens to source control.

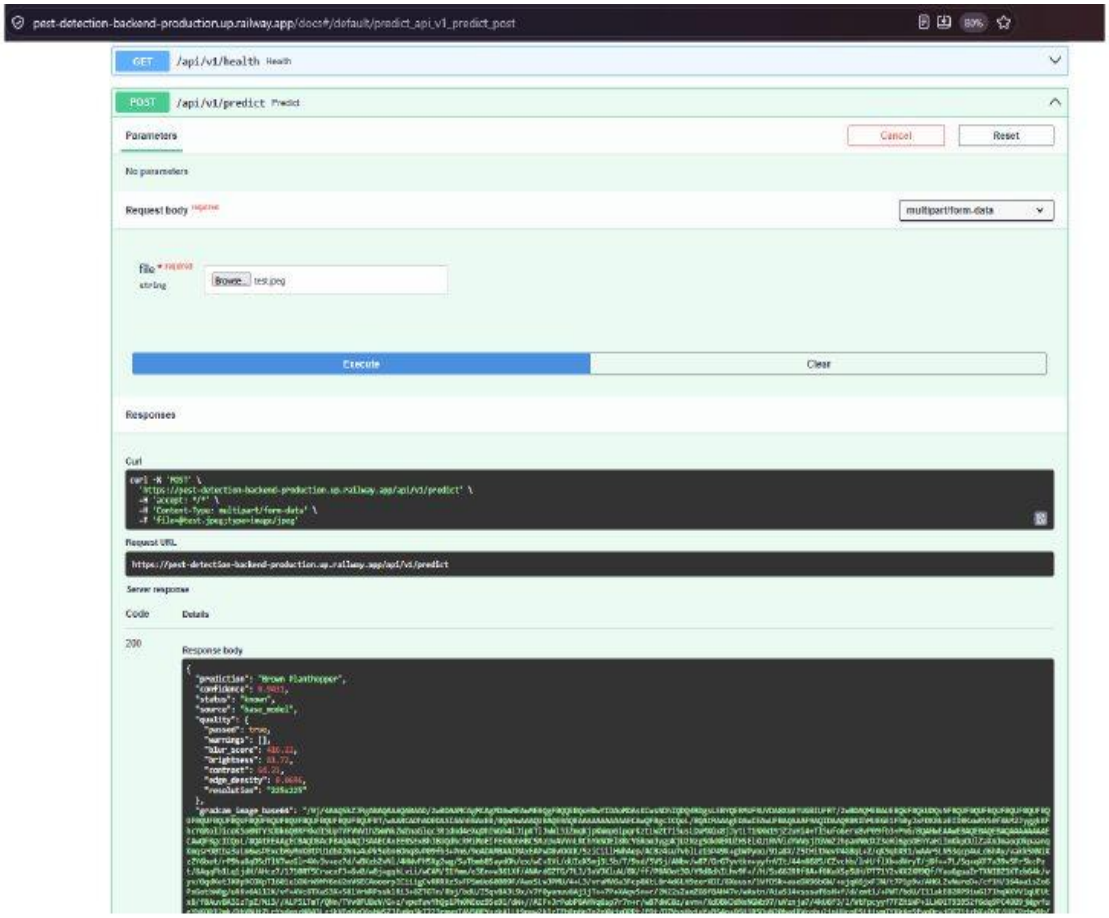
Variable	Purpose
MODEL_PATH	Path to the DenseNet121 state-dictionary checkpoint.
IMAGE_SIZE	Square image resize dimension.
MAX_UPLOAD_SIZE_MB	Maximum permitted uploaded image size.
UNKNOWN_THRESHOLD	Declared unknown threshold setting; verify its use in the deployed code path.
MAYBE_THRESHOLD	Confidence level below which learned-class matching is attempted.
MIN_WIDTH, MIN_HEIGHT	Minimum image dimensions.
BLUR_THRESHOLD	Minimum Laplacian variance for blur check.
MIN_BRIGHTNESS, MAX_BRIGHTNESS	Allowed brightness range.
MIN_CONTRAST	Minimum grayscale contrast.
MAX_EDGE_DENSITY	Maximum edge density before clutter warning.
ALLOWED_ORIGINS	Comma-separated CORS origins.
FEW_SHOT_STORE_DIR	Prototype storage directory.
FEW_SHOT_SIMILARITY_THRESHOLD	Prototype similarity threshold.
FINE_TUNED_STORE_DIR	Fine-tuned adapter storage directory.
FINE_TUNE_EPOCHS, FINE_TUNE_LEARNING_RATE	Fine-tuning training settings.
FINE_TUNE_CONFIDENCE_THRESHOLD	Confidence threshold for adapted-model matching.
OOD_REFERENCE_PATH	Path to the OOD reference tensor file.
OOD_MAHALANOBIS_THRESHOLD	Mahalanobis-distance threshold.

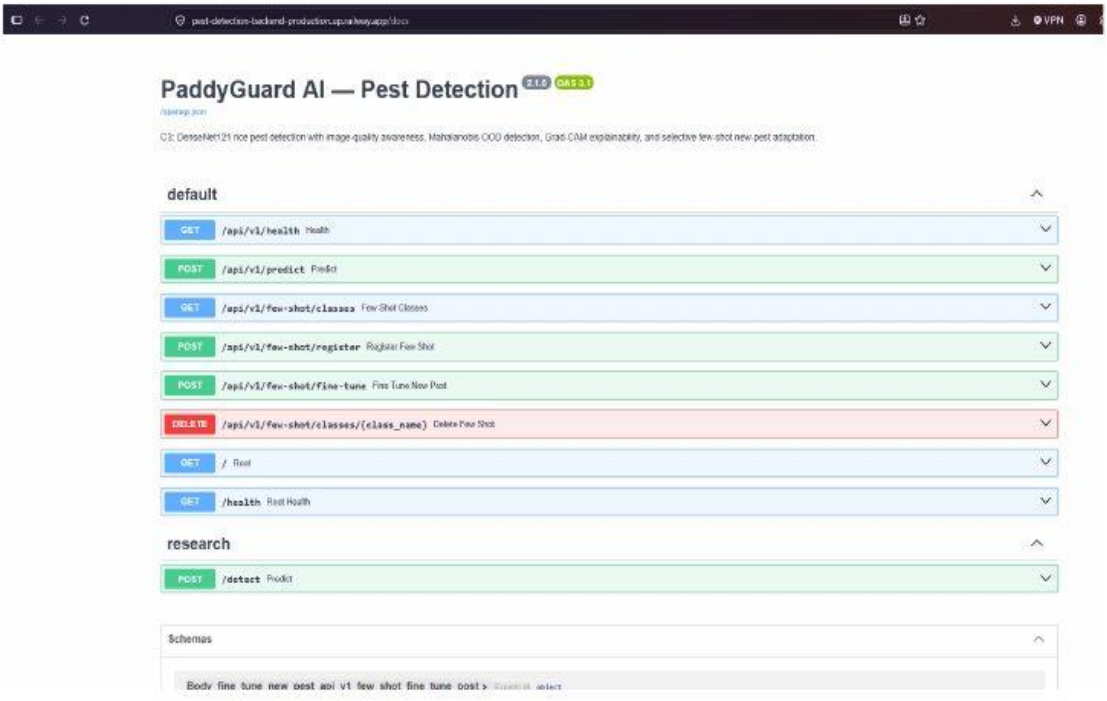
Appendix C – Evidence Screenshots to Insert

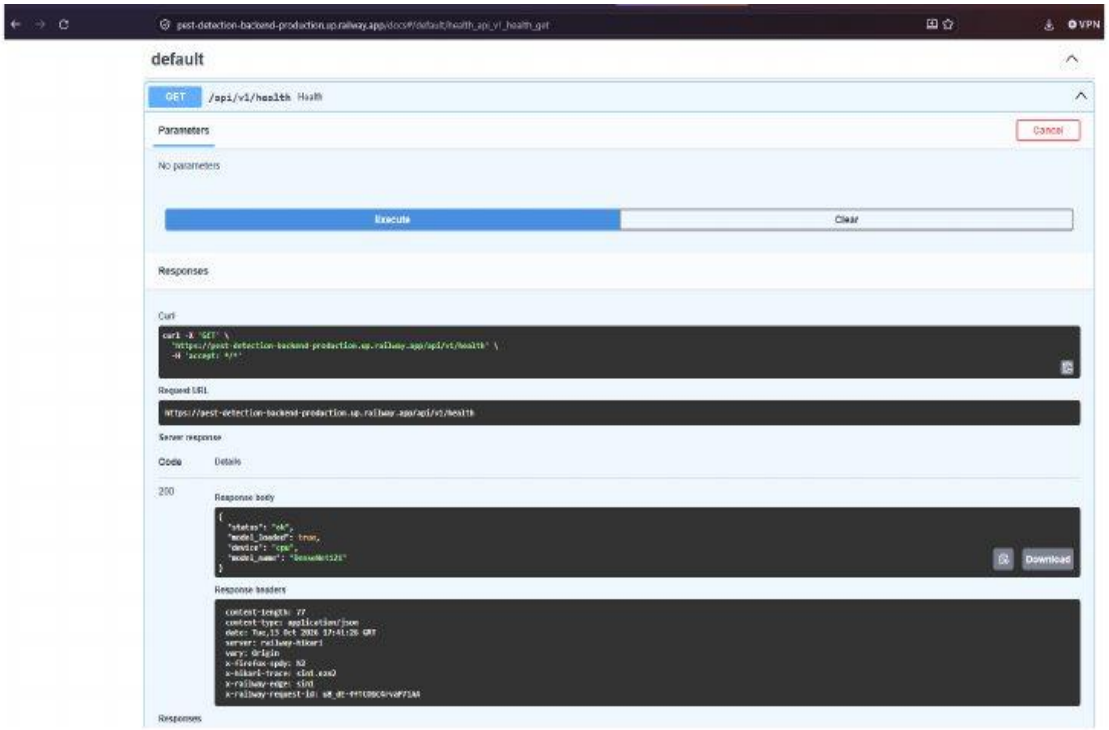
This appendix presents screenshots documenting the deployment and evaluation of the Rice Pest Detection component. The figures cover the Railway deployment environment, API availability, pest prediction responses, image-quality assessment, Grad-CAM visualisation, few-shot learning, fine-tuning, and model evaluation. Each screenshot should represent an actual result obtained from the implemented system.

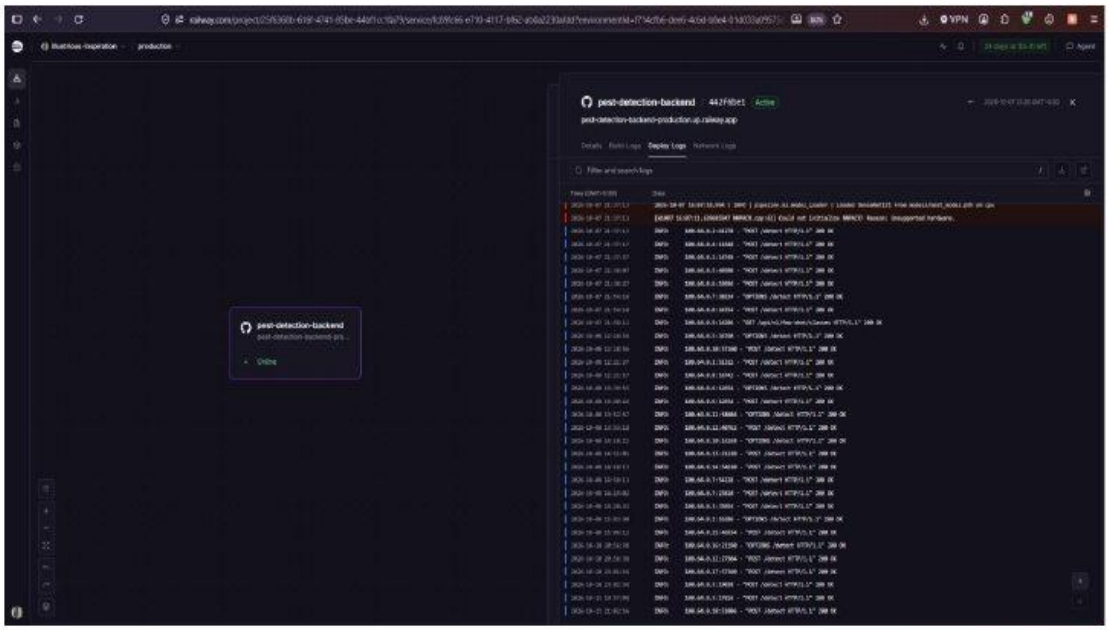


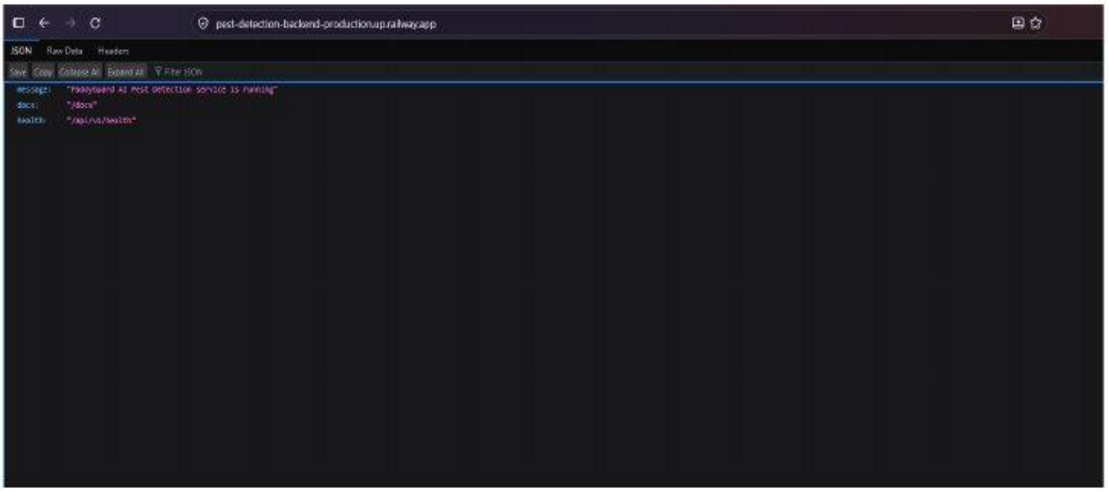


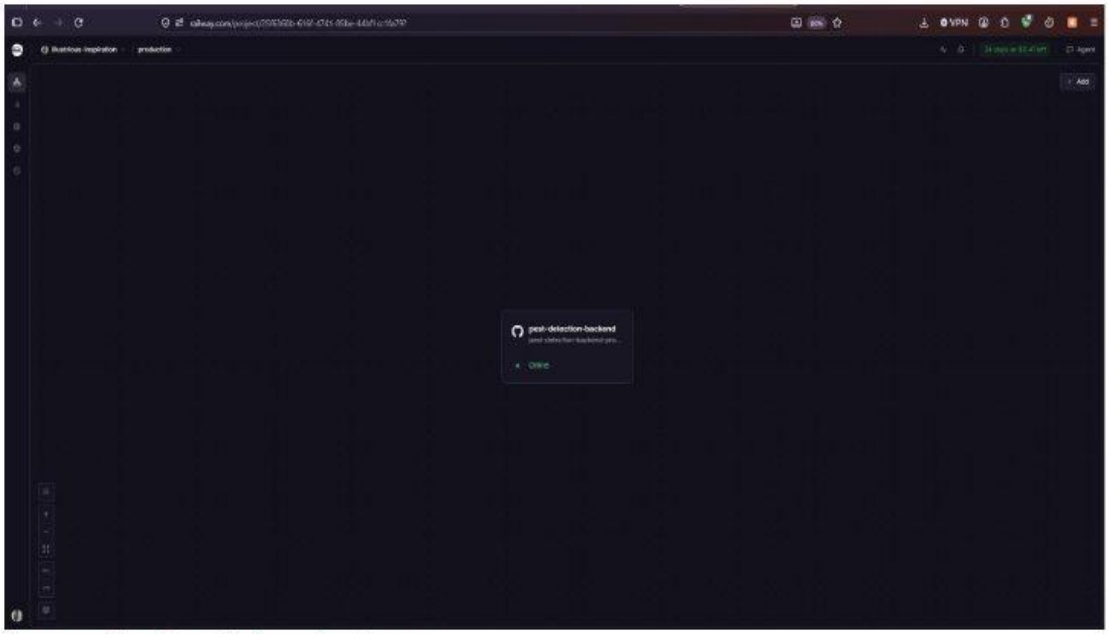

pest-detection-backend-production.up.railway.app/docs#/default/predict_api_v1_predict_post
GET /api/v1/health Health
POST /api/v1/predict Predict
Parameters
Cancel
Reset
No parameters
Request body required
multipart/form-data
file * required
string
Browse... test.jpeg
Execute
Clear
Responses
Curl
curl -X 'POST' \
'https://pest-detection-backend-production.up.railway.app/api/v1/predict' \
-H 'accept: */*' \
-H 'Content-Type: multipart/form-data' \
-F 'file=@test.jpeg;type=image/jpeg'
Request URL
https://pest-detection-backend-production.up.railway.app/api/v1/predict
Server response
Code
Details
200
Response body
"prediction": "Brown Planthopper",
"status": "known",
"source": "base_model",
"quality": {
"passed": true,
"warnings": [],
"blur_score": 410.22,
"brightness": 83.77,
"contrast": 60.21,
"edge_density": 0.0636,
"resolution": "225x225"


pest-detection-backend-production.up.railway.app/docs
VPN
PaddyGuard AI — Pest Detection
2.1.0
OAS 3.1
/openapi.json
C3: DenseNet121 rice pest detection with image-quality awareness, Mahalanobis OOD detection, Grad-CAM explainability, and selective few-shot new-pest adaptation.
default
GET /api/v1/health Health
POST /api/v1/predict Predict
GET /api/v1/few-shot/classes Few Shot Classes
POST /api/v1/few-shot/register Register Few Shot
POST /api/v1/few-shot/fine-tune Fine Tune New Pest
DELETE /api/v1/few-shot/classes/{class_name} Delete Few Shot
GET / Root
GET /health Root Health
research
POST /detect Predict
Schemas
Body_fine_tune_new_pest_api_v1_few_shot_fine_tune_post


default
GET /api/v1/health Health
Parameters
Cancel
No parameters
Execute
Clear
Responses
Curl
curl -X 'GET' \
'https://pest-detection-backend-production.up.railway.app/api/v1/health' \
-H 'accept: */*'
Request URL
https://pest-detection-backend-production.up.railway.app/api/v1/health
Server response
Code
Details
200
Response body
{
"status": "ok",
"model_loaded": true,
"device": "cpu",
"model_name": "DenseNet121"
}
Download
Response headers
content-length: 77
content-type: application/json
date: Tue,15 Oct 2026 17:41:26 GMT
server: railway-edge1
vary: Origin
x-firefox-spdy: h2
x-railway-edge: sin1
Responses


pest-detection-backend
pest-detection-backend-production.up.railway.app
Deploy Logs
pest-detection-backend
Online


pest-detection-backend-production.up.railway.app
JSON
Raw Data
Headers
Save
Copy
Collapse All
Expand All
Filter JSON
message: "Paddyguard AI Pest detection service is running"
docs: "/docs"
health: "/api/v1/health"



Appendix D – Abbreviations

Abbreviation	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
CNN	Convolutional Neural Network
CORS	Cross-Origin Resource Sharing
Grad-CAM	Gradient-weighted Class Activation Mapping
HTTP	Hypertext Transfer Protocol
OOD	Out-of-Distribution
REST	Representational State Transfer
RGB	Red, Green, Blue
CNN	Convolutional Neural Network
UI	User Interface

5. Component 4 — Treatment Advisory System

Original report reproduced in full, without summarization or omission.

PaddyGuard AI

A Multimodal AI-Based System for Rice Leaf Disease & Pest Detection, Treatment Recommendation and Knowledge Assistance

DEVELOPMENT REPORT

Component 4: Treatment Advisory System and Agricultural Chatbot using Retrieval-Augmented Generation

Project ID	R26-SE-015
Student	Keshan B K
Student ID	IT22303820
Degree	BSc (Hons) Degree in Information Technology
Specialization	Software Engineering
Department	Department of Information Technology, Faculty of Computing
Institution	Sri Lanka Institute of Information Technology (SLIIT), Sri Lanka
Supervisor	Dr. Nathali Bagya
Co-supervisor	Ms. Poorna
Report Date	17 October 2026
Status	Draft for supervisor review

Submitted as a development-progress report for the undergraduate research project.

This report documents the author's own component (C4) only; other components belong to other group members.

Declaration

I declare that this development report documents the current state of my individual component, the Treatment Advisory System and Agricultural Chatbot (Component 4) of PaddyGuard AI. Material derived from the approved project proposal and cited sources is acknowledged in the references. Results described as completed are limited to work that has been implemented, observed or evaluated at the time of writing; items without supporting evidence are marked as not tested, unverified or planned.

The backend service, evaluation harness and deployment of C4 are my work. The gateway, the voice, leaf disease and pest components, user management and the shared frontend belong to other group members and appear only at their interfaces with C4. A development report written by another group member was read only to understand the expected structure and level of detail; no text, diagram, screenshot, code or component-specific conclusion from it is used.

An AI assistant (Claude, Anthropic) was used to read the supplied source code, notebook, screenshots and documents, and to draft and restructure this report, its diagrams and tables, with statements tied to the supplied evidence. I remain responsible for checking all content, figures and references against the originals before submission. [AUTHOR TO COMPLETE: describe any other AI tools used during code development.]

Student: Keshan B K Student ID: IT22303820

Signature: _____ Date: _____

Supervisor acknowledgement (if required by the department): Dr. Nathali Bagya Signature: _____ Date: _____

Acknowledgement

I thank my supervisor, Dr. Nathali Bagya, and co-supervisor, Ms. Poorna, for their guidance and feedback on this component. I also acknowledge the other members of the PaddyGuard AI research group, whose individual components, the gateway and the shared frontend make up the integrated platform in which C4 operates.

Executive Summary

This report presents the development and deployment status of Component 4 (C4) of PaddyGuard AI, the Treatment Advisory System and Agricultural Chatbot. C4 is a Node.js and Express service that answers questions about eight rice diseases and pests in English and Sinhala. It decides whether a question is in scope, embeds the question through a Modal-hosted BGE model, retrieves matching passages from a Pinecone index, and asks a language model (`openai/gpt-4o-mini` by default, reached through OpenRouter) to answer from the retrieved text. A second call extracts chemical options so that the web interface can show a dosage calculator.

The service was deployed on Railway (service `paddyguard-treatment-advisory-chatbot`, environment `production`). The deployment record `a1a12349` of 8 October 2026 shows the container starting with `npm start` and `node server.js`, the service listening on port 8080, the Modal embedding provider being selected and its warm-up completing in 15.0 s, and the dashboard shows the service Online with an Active, successful deployment. The Pinecone index `rice-disease-advisor` held 1,271 records at the time of the screenshots. The deployment is not proof that the chatbot works: no recorded test of the public endpoints, of production retrieval or of an end-to-end farmer workflow was supplied, the origin and pooling compatibility of the stored vectors are unresolved, the deployed model variables are masked, and pesticide guidance has not been validated by an expert. The report therefore separates implemented functionality, observed deployment evidence, locally evaluated results and pending work.

Table of Contents

1. Introduction **5**
1.1 Background 5
1.2 Research Problem and Gap 5
1.3 Scope of This Development Report 5
2. Objectives and Research Requirements **6**
2.1 Main Objective 6
2.2 Specific Objectives 6
2.3 Requirements and Implementation Status 6
3. Development Methodology and Tools **7**
3.1 Research and Engineering Approach 7
3.2 Iterative Development Strategy 7
3.3 Technology Stack and Design Decisions 7
3.4 Proposal versus Implementation 8
4. System Architecture **8**
4.1 Overview and Component Boundaries 8
4.2 Runtime Dependencies and Failure Behaviour 9
4.3 Data and State 10
4.4 API Integration 10
5. Component 4 Implementation **10**
5.1 Backend Structure 10
5.2 Knowledge Base and Ingestion 10
5.3 Embedding Generation and Model Configuration 11
5.4 Pinecone Semantic Retrieval 12
5.5 RAG Workflow and LLM Integration 12
5.6 Prompt Design, Language Handling and Safety Behaviour 13
6. Cloud Deployment and Integration **13**
6.1 Railway Service and Build 13
6.2 Deployment Evidence and Runtime Startup 14
6.3 Configuration and Secrets 15
6.4 Frontend and Gateway Integration 15
7. Testing and Current Results **16**
7.1 Verification Register 16
7.2 Unit and Evaluation Tests 16
7.3 Interpretation 17
8. Challenges and Resolutions **17**
9. Current Status and Remaining Work **18**
9.1 Immediate Next Development Steps 18
10. Research Evaluation Plan **19**
10.1 Test Discipline 19
10.2 Retrieval and Answer Evaluation 19
10.3 Deployment and Usability Evaluation 19
10.4 Security and Operational Improvements 20
11. Conclusion **20**

List of Abbreviations

Abbreviation	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
BGE	BAAI General Embedding
C1-C4	Components of PaddyGuard AI; C4 is the Treatment Advisory System and Agricultural Chatbot
CLS	Classification token of a BERT-style model
CORS	Cross-Origin Resource Sharing
CPU	Central Processing Unit
FAO	Food and Agriculture Organization
HTTP	Hypertext Transfer Protocol
IRAC	Insecticide Resistance Action Committee
IRRI	International Rice Research Institute
JSON	JavaScript Object Notation
LLM	Large Language Model
NLP	Natural Language Processing
ONNX	Open Neural Network Exchange
RAG	Retrieval-Augmented Generation
RAM	Random-Access Memory
SDK	Software Development Kit
UI	User Interface

1. Introduction

1.1 Background

PaddyGuard AI is a group research project that combines Sinhala voice input, leaf disease classification, pest detection, a treatment advisory chatbot and user management behind a single API gateway [1]. Detection tools often stop at the diagnosis, whereas a farmer also needs to know what to do next, and general-purpose chatbots can state unsupported facts about pesticides. Component 4 (C4) is the part of the platform that turns a diagnosis, or a free-text question, into treatment guidance grounded in a curated set of agricultural documents.

Component numbering differs between project documents. The progress-presentation slides call the treatment component C3, whereas the final architecture, the gateway and the service itself (`GET /health` returns `component: "C4"`) call it C4. This report uses C4. The gateway, the voice, leaf and pest components and user management belong to other group members and are described only where C4 depends on them.

1.2 Research Problem and Gap

The proposal identifies three practical limitations that motivate C4: many farmers have limited access to agricultural experts, disease-detection tools rarely continue beyond the diagnosis, and unconstrained language models may give treatment advice that is not traceable to a source [1]. The development problem was therefore to build a service that answers rice disease and pest questions from a curated knowledge base, works in English and Sinhala, accepts input from other components and can run as a hosted service [2].

> **Development focus**
>
> This report concentrates on what the supplied evidence shows about the implemented backend and its deployment. It does not claim that treatment advice is clinically or agronomically validated, and it does not treat a successful Railway deployment as proof that the chatbot has been tested end to end.

1.3 Scope of This Development Report

The author's contribution comprises the Node.js/Express advisory service, the retrieval-augmented generation (RAG) pipeline, the eight-topic taxonomy and scope gate, chemical-option extraction, the in-memory session store, the Jest tests and evaluation harness, the Dockerfile, the Modal embedding service, the Kaggle notebook used for knowledge-base preparation, the Railway deployment, and the treatment chat page and dosage calculator inside the shared frontend [2]. The gateway, user management and components C1 to C3 are the work of other members.

Each claim is tied to one of the evidence types in Table 1. A screenshot of a configuration is never treated as proof that a workflow works. Four limits apply to the evidence: the GitHub repository could not be opened during the review, so commit history was not inspected (the supplied ZIP snapshot has no README and no `.git` directory); the Railway build log is available only as its final lines; environment-variable values are masked; and screenshot dates come from the author's browser clock.

Table 1. *Evidence status labels used in this report*

Label	Meaning
Source-implemented	Present in the source snapshot of the backend, frontend or gateway; says nothing about whether it was run.
Observed	Visible in a log or screenshot; a single observation, not a test.
Author-reported	Stated by the author without a supplied output.
Evaluated	Measured by the C4 evaluation harness in the individual thesis [2].
Not tested / Pending	No evidence supplied, or work not yet done.

2. Objectives and Research Requirements

2.1 Main Objective

To design, implement and deploy a retrieval-augmented treatment advisory service for eight rice diseases and pests that answers in English and Sinhala, integrates with the PaddyGuard AI gateway and frontend, and records transparently which parts have been verified.

2.2 Specific Objectives

1. **D1.** Implement a retrieval-augmented advisory service with a documented API.
2. **D2.** Build and use a vector knowledge base covering eight rice diseases and pests.
3. **D3.** Support English and Sinhala questions and answers.
4. **D4.** Integrate with the gateway and the frontend of the group platform.
5. **D5.** Deploy the service on a hosted platform with environment-based configuration.
6. **D6.** Record what was verified and what remains unverified.

2.3 Requirements and Implementation Status

The proposal lists nine functional requirements [1]. Table 2 records their status using the source snapshot as the primary evidence.

Table 2. *Functional requirements and implementation status*

Requirement	Status in the C4 service	Evidence
FR1 Query submission	Implemented: `POST /chat`.	`routes/chat.js`
FR2 Image upload	Not part of C4; handled by the leaf and pest components.	Architecture
FR3 NLP processing	Language detection, alias matching and LLM normalisation of Sinhala queries; no separate tokenisation or keyword-extraction module.	`languageService.js`, `queryNormalizer.js`
FR4 Knowledge retrieval	Implemented: query embedding and Pinecone vector query.	`retriever.js`
FR5 Recommendations	Implemented through LLM generation from retrieved text.	`chatService.js`
FR6 Rule-based validation	Partial: topic taxonomy, keyword scope gate and prompt rules; no rule engine.	`diseasesPests.js`, `scopeService.js`
FR7 Chatbot responses	Implemented in English and Sinhala.	`chatService.js`
FR8 Preventive advice	Produced when the retrieved text supports it and the prompt asks for it.	System prompt
FR9 Knowledge-base management	Not in the service; ingestion is an offline notebook procedure.	Kaggle notebook

Non-functional requirements follow the quality headings of ISO/IEC 25010 [3]. For performance, the proposal's short-response target was not met in the local evaluation (mean 10.2 s against an 8 s target) [2]. For reliability and security the evidence is partial: the route handles missing fields and errors, a warm-up failure does not stop the server, secrets are read from environment variables, and the Modal endpoint requires a bearer token, but C4 has no authentication or rate limit of its own and sessions are lost on restart. Usability has screenshots but no user study, and maintainability is supported by the modular code layout and the presence of test files.

3. Development Methodology and Tools

3.1 Research and Engineering Approach

C4 was developed incrementally and notebook-first: retrieval and prompting ideas were tried in a Kaggle notebook, then ported to a Node.js service whose modules state in code comments that they mirror the notebook functions. The pipeline is split into stages that can be tested separately (language detection, scope check, query normalisation, embedding, retrieval, generation, chemical extraction), and `server.js` exports the Express app and listens only when run directly, which allows route tests with supertest. The evaluation harness calls the service function directly, so retrieval and generation can be measured without HTTP, the gateway or the frontend.

3.2 Iterative Development Strategy

Table 3 reconstructs the development sequence from the available evidence. The order of the first two steps is inferred from source comments describing the service as a port of the notebook.

Table 3. *Reconstructed development chronology*

Step	Activity	Date or evidence	Status
1	Notebook prototype: ingestion code, retrieval tests, chat loop	Notebook and screenshots	Observed
2	Port to a Node.js/Express service with Jest tests	Backend source snapshot	Source-implemented
3	Evaluation runs and root-cause analysis	6 and 7 Oct 2026, thesis [2]	Evaluated
4	Modal embedding service added on branch `feature/modal-bge-embedding`, merged as pull request #1	Railway deployment titles	Observed (titles only)
5	Railway deployment `a1a12349`: container started, Modal provider warmed	8 Oct 2026, 01:20 (GMT+5:30)	Observed
6	Public frontend: one English, then one Sinhala interaction	10 and 11 Oct 2026	Observed (single)
7	Railway shows the service Online with an Active deployment	16 and 17 Oct 2026	Observed

3.3 Technology Stack and Design Decisions

Table 4 lists the technologies with the versions declared in the source. Installed versions on Railway were not recorded; the start log shows only the package name and version `treatment-advisory-chatbot@2.0.0`.

Table 4. *Implemented technology stack of the C4 backend*

Layer	Technology (declared in source)	Role and recorded rationale
Runtime	Node.js 20 (`node:20-slim`) [4]	Runs the Express service.
Web framework	Express ^4.19.2, cors, dotenv [5]	Routes, CORS and environment loading.
LLM access	`openai` SDK ^4.47.1 pointed at OpenRouter [6]	One client for all LLM calls; code comment says it mirrors the notebook client.
Vector database	Pinecone client ^3.0.3 [7]	Semantic retrieval from index `rice-disease-advisor`; read-only from the service.
Query embedding	Modal app `paddyguard-bge-embedder`: Python 3.11, torch 2.5.1, transformers 4.47.1, FastAPI [8]; model BAAI/bge-large-en-v1.5 [9]	Production path. A source comment states that local inference needs about 1-1.5 GB RAM in the container.
Embedding (alternative)	`@xenova/transformers` ^2.17.2 (ONNX)	Local query embeddings when the provider is `local` (the code default).
Testing	Jest ^29.7.0, supertest ^7.0.0, custom evaluation harness	Unit, route and evaluation tests.
Hosting	Railway (service) [10], Modal (embedding app)	Deployment of the service and of the embedding model.

Four trade-offs are the author's assessment rather than recorded decisions. Moving query embedding to Modal keeps the Railway container small but adds a network call, a cold start (the Modal app scales down after 300 s) and a second secret. Plain SDK calls instead of a framework such as LangChain give full control of prompts and few dependencies, at the cost of more custom code. Sessions held in a process-level `Map` are simple and fast but are lost on restart and cannot be shared by two instances. Handling Sinhala by normalising queries to English reuses an English-only embedding model, but adds an LLM call and a risk of translation error.

3.4 Proposal versus Implementation

The proposal named MongoDB Atlas Vector Search as the knowledge store, LangChain or Hugging Face Transformers as the NLP framework, a rule engine to validate recommendations, and IRRI and FAO documents as sources [1]. The implemented service differs: retrieval uses a Pinecone index, LLM and embedding calls use the SDKs directly, validation is limited to the taxonomy, scope gate and prompt rules, and no IRRI or FAO file name was observed in the index metadata. The deployed code declares no MongoDB, PostgreSQL or Redis dependency in `package.json`. A comment in `sessionStore.js` says the service once shipped with a Mongo-backed session library; this indicates an earlier design, not a deployed one, and MongoDB is not part of the C4 implementation described here.

4. System Architecture

4.1 Overview and Component Boundaries

A React frontend calls the FastAPI gateway, which forwards chat requests to C4. Table 5 states where C4 sits and which parts belong to other members.

Table 5. *Component boundaries around C4*

Component	Responsibility	Implementation / owner
Frontend	User interface; treatment chat page, dosage calculator and advisor handoff are C4 contributions	React, Vite, TypeScript; shared
Gateway	Public API entry point; maps `/api/v1/chat/*` to C4	FastAPI; other member
C1 to C3, user management	Voice, leaf disease and pest components and accounts; C4 receives only a diagnosis-derived first message	Other members
C4 service (this report)	Scope check, retrieval, answer generation, chemical extraction, sessions	Node.js/Express on Railway
External services used by C4	Query embeddings; vector retrieval; LLM completions	Modal; Pinecone; OpenRouter

Figure 1 shows the internal structure. The route module receives requests from the gateway, `chatService.js` orchestrates the pipeline, the embedder, retriever and LLM client wrap the three external services, and the taxonomy and session modules hold the only local state.

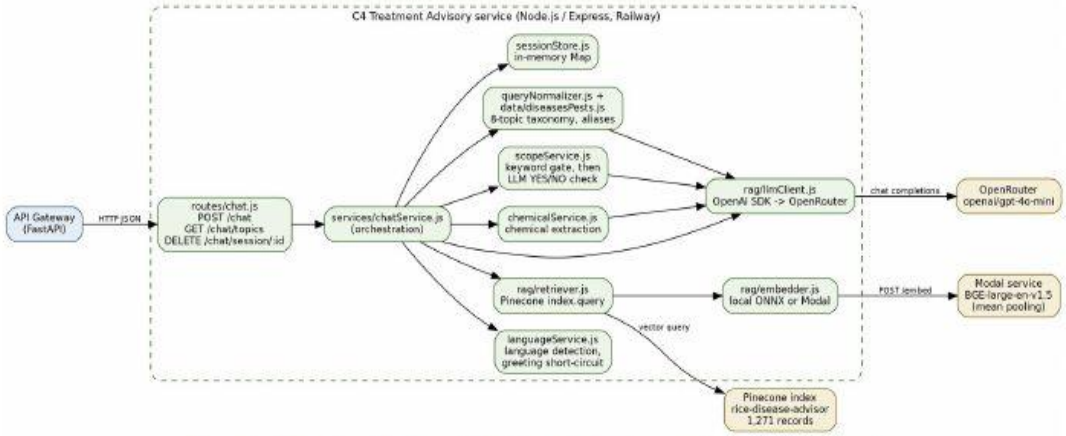


Figure 1. *Internal architecture of the C4 service (author-created from the source)*

4.2 Runtime Dependencies and Failure Behaviour

Table 6. *Runtime dependencies of the C4 service (variable names only)*

Dependency	Protocol and authentication	Configuration names	Behaviour if unavailable (source)
Modal embedder	HTTPS `POST /embed`, bearer token	`MODAL_EMBEDDING_URL`, `MODAL_AUTH_TOKEN`, `MODAL_TIMEOUT_MS`	Error thrown and route returns HTTP 500; a warm-up failure is logged and the server keeps running.
Pinecone	Pinecone client, API key, optional host	`PINECONE_API_KEY`, `PINECONE_HOST`, `PINECONE_INDEX_NAME`	Missing key throws; query errors propagate to HTTP 500.
OpenRouter	OpenAI-compatible HTTPS API, API key	`OPENROUTER_API_KEY`, `OPENROUTER_BASE_URL`, `LLM_MODEL`	Generation errors give HTTP 500; extraction failure returns an empty chemical list; follow-up failure is logged.
Gateway (caller)	HTTP JSON; no authentication inside C4	`ALLOWED_ORIGINS` for browser CORS	Gateway maps timeouts and connection errors to 504 and 503.

4.3 Data and State

C4 owns no relational or document database. Persistent knowledge lives in the Pinecone index, which the service only reads: `retriever.js` calls `index.query`, and nothing in the service upserts vectors. Conversation state is a JavaScript `Map` keyed by `session_id` inside the Node.js process, and the taxonomy of eight topics with English and Sinhala aliases is a source file (`data/diseasesPests.js`). This is the complete data architecture evidenced by the source.

4.4 API Integration

Table 7. *Routes exposed by the C4 service and their evidence status*

Route	Purpose	Status
`GET /health`	Liveness information, returns `component: "C4"`	Source-implemented; not tested on the deployed domain
`POST /chat`	Answer a question; body `{message, session_id}`	Source-implemented; not tested on the deployed domain
`GET /chat/topics`	List the eight supported topics	Source-implemented; not tested
`DELETE /chat/session/:sessionId`	Clear a conversation	Source-implemented; not tested

A request contains `message` and `session_id`, both required; there is no language parameter, because the service infers the language from the characters in the message. Validation is limited to the presence of the two fields (HTTP 400 otherwise), unexpected errors return HTTP 500 with a `detail` field, and there is no length limit beyond Express's default body limit. The reply fields are listed in Appendix A. In the gateway source, the chat router forwards `/api/v1/chat/message`, `/topics` and `/session/:id` to `{TREATMENT_CHATBOT_URL}/chat`, `/chat/topics` and `/chat/session/{id}`, relays the status code and JSON, and returns 504 on a timeout and 503 when C4 cannot be reached (timeouts of 60 s for the message call and 30 s for the others). In the gateway snapshot the authentication middleware and rate limiter are not registered, so the chat route is public; whether the deployed gateway forwards successfully was not tested.

5. Component 4 Implementation

5.1 Backend Structure

The application code is about 1,340 lines. `server.js` sets up Express, CORS, JSON parsing, `/health`, route mounting and the embedder warm-up. `config/index.js` holds every environment-driven setting and its default, in line with the practice of keeping configuration in the environment [11]. `routes/chat.js` exposes the chat routes. `services/` contains the orchestrator (`chatService.js`, the largest module at 281 lines), language detection, the scope gate, query normalisation, chemical extraction and the session store. `rag/` wraps the embedder, the Pinecone retriever and the OpenRouter client, `data/diseasesPests.js` holds the taxonomy, and `modal/app.py` is the embedding service. Two gaps are noted: the repository has no `.dockerignore`, so the Docker build copies the whole context including tests and evaluation reports, and the snapshot has no README.

5.2 Knowledge Base and Ingestion

The corpus is a Kaggle dataset named `rice-disease-and-pest-KB` (Version 1, 19.82 MB) mounted as a notebook input. The visible files are named like journal articles, topic files (for example on gall midge, hispa and leaf folder) and an insecticide resistance guideline; the first file shown is an article on integrated pest management of basmati rice [12]. The author reports that a verification run found 35 PDF files, all with extractable text; that output was not supplied.

The ingestion code in the notebook extracts text with `pypdf`, splits it with a recursive character splitter (chunk size 800, overlap 120, minimum length 20), encodes chunks with BAAI/bge-large-en-v1.5 with normalisation, and upserts to Pinecone in batches of 64 with UUID identifiers and the chunk text and source file name as metadata. The recorded notebook run skipped ingestion because the index already held 1,271 vectors, so this code documents the intended procedure and does not prove how the existing vectors were made. The service itself never ingests.

Figure 2 shows the Pinecone index `rice-disease-advisor`: 1,271 records, region AWS us-east-1, one namespace (`__default__`) and the index model label `llama-text-embed-v2`. A notebook check of the index shows that all 1,271 records carry `source` metadata, with 35 unique source filenames. Reprocessing the currently available PDFs with the configured splitter gave 1,250 chunks, 21 fewer than the index holds (author-reported). The reason is unresolved and no cause is asserted; the figure describes what the current corpus and parameters yield, not how the existing records were created. No file name identifies IRRI or FAO material, although several names are not descriptive, so the absence of such sources cannot be concluded.

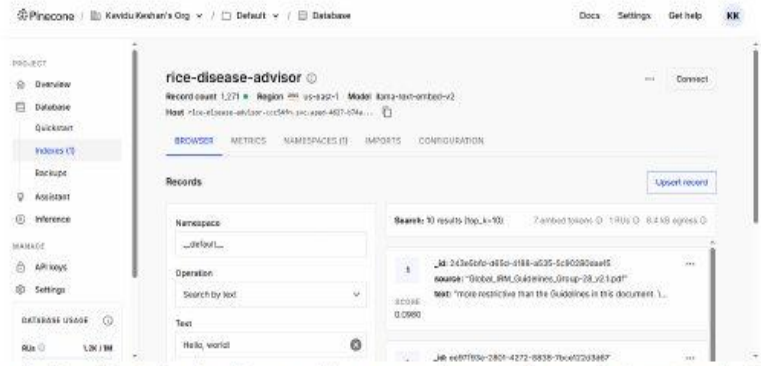


Figure 2. *Pinecone dashboard for index rice-disease-advisor, 17 October 2026 (the search shown is a dashboard text search, not the service query)*

5.3 Embedding Generation and Model Configuration

`rag/embedder.js` selects a provider from `EMBEDDING_PROVIDER` (default `local`); the Railway startup log shows that the `modal` provider was configured. The Modal app downloads the model weights at build time, loads the tokenizer and model once per container, uses 2 CPUs and 4,096 MB of memory, and exposes `GET /health` and `POST /embed`. The embed endpoint requires a bearer token, accepts 1 to 4,096 characters, tokenises to at most 512 tokens, applies attention-masked mean pooling and L2 normalisation, checks for 1,024 values and returns `{embedding, dimension, model}`. When and how the Modal app was deployed is not recorded, and no Modal dashboard screenshot was supplied. The backend client accepts only an array of 1,024 finite, non-zero numbers. At start-up `warmUp()` embeds the text "warmup"; because it awaits a real, validated embedding call, the logged "Modal warm-up complete" message is consistent with one successful authenticated request from the Railway container to Modal. It is not a test of retrieval or generation.

The notebook, the source code, the Railway dashboard and the Pinecone dashboard each show a different model context, and Table 8 keeps them separate rather than combining them into one configuration.

Table 8. *Model configuration by evidence context*

Evidence context	LLM	Embedding model	Status
Kaggle notebook configuration cell	`openai/gpt-4o-mini` via OpenRouter	`BAAI/bge-large-en-v1.5`	Observed (prototype workflow)
Backend source defaults	`LLM_MODEL` default `openai/gpt-4o-mini`	Local default `Xenova/bge-large-en-v1.5`; Modal app fixes `BAAI/bge-large-en-v1.5`	Source-implemented
Railway deployment	Variable names visible, values masked	Log shows provider `modal`; model value not shown	Observed (partial)
Pinecone dashboard	Not applicable	Index model label `llama-text-embed-v2`	Observed; relation to the raw-vector query path unresolved

The service queries by vector using 1,024-dimensional BGE embeddings. It does not use the dashboard text search, which depends on the index model label [7]. A related open issue concerns pooling. The BAAI documentation states that BGE models use the final hidden state of the `[CLS]` token as the sentence embedding [13], whereas both query paths in C4 use mean pooling. If the stored vectors were created with the default CLS pooling, as a SentenceTransformer encoding normally does, queries and documents would not lie in exactly the same space. Retrieval scores of about 0.74 to 0.78 show that related passages are still returned, which is weak evidence of compatibility. A verification script that re-embeds sampled stored records with several methods was prepared with the thesis but has not been executed, so no similarity values are reported.

5.4 Pinecone Semantic Retrieval

`rag/retriever.js` embeds the query, obtains the index handle (with the host if `PINECONE_HOST` is set), calls `index.query` with `topK` and `includeMetadata`, and maps each match to `{text, source, score}`. The default `topK` is 6 (`RETRIEVAL_TOP_K`) and 8 when the message is a bare topic name (`RETRIEVAL_TOP_K_BARE_TOPIC`). The notebook states a cosine metric and 1,024 dimensions. The index host in the dashboard differs from the host in `.env.example`, which suggests the index was recreated after the template was written; the reason was not recorded. Because the deployed values of `PINECONE_HOST` and `PINECONE_INDEX_NAME` are masked, the statement that the deployed service queries this particular index is inferred from the code default and the dashboard, not observed.

Retrieval evidence is limited. In the notebook, a rice blast question returned five results with scores from 0.7786 down to 0.7428 (prototype), while Pinecone text search for the same question gave scores of only 0.044 to 0.055. The evaluation harness, run locally with an unrecorded embedding provider, gave a mean top score of 0.7531 with 9 of 10 cases above 0.70 [2]. Retrieval from the deployed Railway service has not been tested.

5.5 RAG Workflow and LLM Integration

A message first passes language detection (Sinhala Unicode range U+0D80 to U+0DFF) and a greeting check, which returns a fixed reply without retrieval. The scope check accepts the message on a keyword or alias match and otherwise asks the LLM for a YES or NO answer; out-of-scope questions receive a fixed decline message listing the eight topics. For in-scope messages the service builds a retrieval query (Sinhala input is substituted with aliases and translated to English by the LLM, and follow-up topics are resolved from history), embeds it, retrieves chunks from Pinecone and asks the LLM to answer from the chunks and the session history, following the RAG pattern [14]. A second call then extracts chemical options as JSON for the dosage calculator, and a third suggests a follow-up question.

All LLM calls use one OpenAI SDK client whose base URL is `OPENROUTER_BASE_URL` (default `https://openrouter.ai/api/v1`) and whose key is `OPENROUTER_API_KEY`; a missing key only logs a warning

and calls fail later. The model is `LLM_MODEL`, with the code default `openai/gpt-4o-mini`; the deployed value is masked. The OpenRouter workspace screenshot (10 October 2026) shows only GPT-4o-mini usage (1.52K requests, 1.27M tokens, $0.27 in one week), but it is workspace-wide and is not attributed to C4 requests. Table 9 lists the calls made for one request.

Table 9. *LLM calls made by the C4 service (from source)*

Call	When	Temperature	Max tokens	Failure behaviour
Scope classification	Only if no keyword matches	0	5	Not caught; HTTP 500
Query normalisation	Sinhala input	0	35	Falls back to the original query
Answer generation	In-scope messages	0.3	1,500	Not caught; HTTP 500
Chemical extraction	After generation	0	600	Returns an empty list
Follow-up question	After generation	not read	60	Logged; follow-up is null

A typical English in-scope message that matches a keyword therefore makes three LLM calls, and a Sinhala message adds a normalisation call. This is relevant to latency, since the thesis evaluation measured a mean response time of 10.2 s [2].

5.6 Prompt Design, Language Handling and Safety Behaviour

The system prompt in `chatService.js` fixes the role (a rice disease and pest advisor limited to the eight topics), states that scope filtering has already been done, and requires the answer in the language of the user's current message with chemical and scientific names kept in English. Its numbered rules tell the model to answer from the provided context and history, to use its own knowledge only to fill gaps without contradicting the context, to follow the requested format, to give a full report when the message is only a topic name, and to resolve follow-ups from history. Rule 7 is a treatment and dosage safety rule: recommend only practical farmer-facing rates (g/L, mL/L, kg/ha), never output laboratory in-vitro concentrations such as ppm, and say that a verified field dosage is not specified when none exists. Retrieved chunks are formatted into a context block ("No relevant knowledge base context found." if none) and sent with the replayed session history, which is not truncated.

The extraction prompt likewise forbids laboratory concentrations as field doses and sets the dose to "Not specified" when no verified rate exists, but it also allows a reasonable standard estimate prefixed with "Approx." and asks for coverage per litre to be estimated similarly. The frontend calculator scales whatever values result [2]. The prompt rules and keyword scope gate are safeguards, not guarantees: no expert has validated the output, and the Sri Lankan registration status of any recommended active ingredient has not been checked.

Exceptional cases are handled as follows. Unknown session identifiers start a new conversation; a failed Sinhala normalisation logs a warning and uses the original query; empty retrieval produces a context block stating that nothing was found; extraction and follow-up failures are caught and return an empty list or null; and a warm-up failure is logged without stopping the server. Failures of the scope call, embedding, Pinecone query or answer generation are not caught inside the pipeline and surface as HTTP 500. None of these paths has a recorded test result in the supplied deployment evidence.

6. Cloud Deployment and Integration

6.1 Railway Service and Build

The service `paddyguard-treatment-advisory-chatbot` runs in the Railway project `handsome-renewal`, environment `production`, with one replica in the US East region and the public domain `paddyguard-treatment-advisory-chatbot-production.up.railway.app`. Figure 3 shows the dashboard on 17 October 2026: status Online and one Active deployment marked "Deployment successful". The deployment history lists four entries, three of them Removed. Each entry carries the GitHub mark and "via GitHub", and the titles are "feat: add treatment

advisory chatbot" (twice) and "Merge pull request #1 from kavi20002/feature/modal-bge-embedding" (twice, one Active); why the merge title appears twice is not shown. The source repository is https://github.com/kavi20002/paddyguard-treatment-advisory-chatbot.

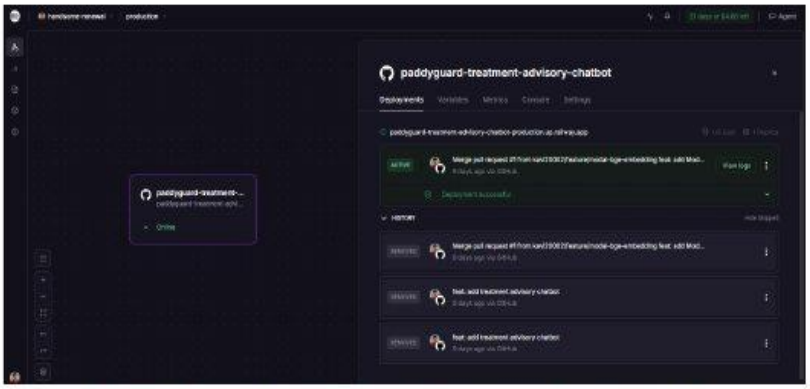


Figure 3. *Railway deployments tab for the C4 service, 17 October 2026*

The repository Dockerfile builds from `node:20-slim`, sets `/app` as the working directory, copies `package*.json`, runs `npm install --omit=dev`, copies the source, declares `EXPOSE 8004` and starts with `npm start` [15]. The final lines of the 8 October 2026 build log show a cached `WORKDIR /app` step at 01:20:27, export to Docker image format, an image digest at 01:20:29 and an image push of 213.9 MB at 01:20:30. This supports a successful image build and push for that deployment; it does not show dependency installation output or the total build time. The Dockerfile declares port 8004 while the log reports port 8080. `EXPOSE` documents a port and does not fix it [15], the service listens on `process.env.PORT` and defaults to 8004, and Railway supplies a port to the container [10], so the difference is consistent with a platform-assigned `PORT`; because that variable is not visible in the truncated variables list, this remains an inference. The dashboard also shows a trial credit indicator (21 days or $4.86 left on 17 October 2026). Build and start command overrides, health-check path, restart policy and resource limits are not evidenced because the Settings tab was not supplied.

6.2 Deployment Evidence and Runtime Startup

Figure 4 shows the deploy log of deployment `a1a12349`, status Active, dated 8 October 2026 at approximately 01:20 (GMT+5:30), and Table 10 relates each line to the code that produces it.

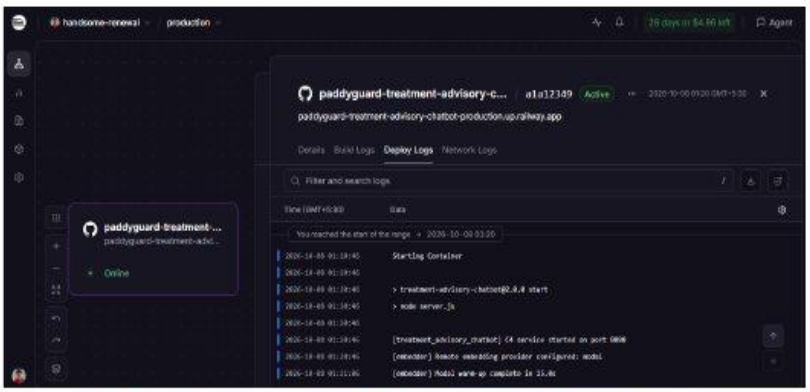


Figure 4. *Railway deploy log of deployment a1a12349, 8 October 2026*

Table 10. *Interpretation of the startup log*

Log line (time)	Produced by	What it shows	What it does not show
Starting Container (01:20:46)	Railway	The container started	That the application is healthy
`> treatment-advisory-chatbot@2.0.0 start; > node server.js`	`npm start`, `package.json`	Package name, version and entry point ran	Installed dependency versions
C4 service started on port 8080	`server.js` listen callback	Express is listening	That any route returns correct data
Remote embedding provider configured: modal	`embedder.js warmUp()`	`EMBEDDING_PROVIDER` was `modal`	The model or endpoint value
Modal warm-up complete in 15.0s (01:21:06)	`embedder.js` after `embedQuery`	One embedding request to Modal succeeded and was validated	Retrieval, generation or any API test

One timing detail is noted without explanation: the provider message and the completion message are 20 seconds apart in the log (01:20:46 and 01:21:06), whereas the application reports an elapsed time of 15.0 s. Taken together, the evidence supports that the service was built, started, listened on port 8080 and reached its embedding provider on 8 October 2026, and that Railway reported it Online with an Active deployment on 16 and 17 October. It does not show that the public endpoints respond correctly, that production retrieval works, that the deployed LLM is `openai/gpt-4o-mini` (the variable is masked), or that the Sinhala workflow works in production.

6.3 Configuration and Secrets

The Railway variables list shows the names `ALLOWED_ORIGINS`, `EMBED_MODEL_NAME`, `EMBEDDING_PROVIDER`, `LLM_MODEL`, `MODAL_AUTH_TOKEN`, `MODAL_EMBEDDING_URL`, `MODAL_TIMEOUT_MS`, `OPENROUTER_API_KEY`, `OPENROUTER_BASE_URL`, `PINECONE_API_KEY`, `PINECONE_HOST`, `PINECONE_INDEX_NAME`, `RETRIEVAL_TOP_K` and `RETRIEVAL_TOP_K_BARE_TOPIC` with values masked; the list is cut off at the bottom of the panel, so `PORT` is not visible. Appendix B gives the code defaults, which are in the public source and are not proof of the deployed values. Existing controls are environment-based configuration, a `.gitignore` entry for `.env`, masked values in Railway, a bearer token on the Modal endpoint, and platform secrets for the Kaggle notebook. Weaknesses are that `ALLOWED_ORIGINS` defaults to `*` when unset, that C4 has no authentication or rate limiting, that message text is logged when normalisation fails, and that the frontend and gateway archives supplied for review contain `.env` files that were not opened; if they hold real keys they should be rotated. No key or token value appears in this report.

6.4 Frontend and Gateway Integration

The frontend calls `POST /api/v1/chat/message`, `GET /api/v1/chat/topics` and `DELETE /api/v1/chat/session/:id` on the gateway, and the mapping to C4 is established at source level (Section 4.4). Because the request carries no language field, the reply language follows the characters in the message. A screenshot of the public frontend on 10 October 2026 shows an English message about Brown Planthopper treatment and an English reply, and a screenshot from 11 October 2026 (21:51) shows a Sinhala question and the opening of a Sinhala reply. In the latest frontend source, the handoff from a diagnosis result builds the first message in Sinhala when the interface language is Sinhala and starts a new session. These are single observations: they do not show the rest of the reply, the gateway route used or any scripted test, and the correctness of the replies was not assessed. The screenshots are not reproduced in this report.

7. Testing and Current Results

7.1 Verification Register

Table 11 records each verification activity. An activity is marked as passed only when a result was recorded; deployment-level checks are limited to observations, and no functional test of the public service has been recorded.

Table 11. *Verification register*

Activity	Observed result	Method	Status
Build success	Export and push of 213.9 MB in the log tail	Railway build log	Observed (partial)
Container start and port	`npm start`, `node server.js`, port 8080	Deploy log	Observed
Deployment status	Active, "Deployment successful"	Dashboard	Observed
Embedding service reachable	Warm-up complete in 15.0 s	Start-up warm-up	Observed
Public availability and `/health`	No request recorded	HTTP request	Not tested
`POST /chat`, `/chat/topics`, `DELETE`	No request recorded	HTTP requests	Not tested
Vector retrieval (prototype)	Scores 0.7786 to 0.7428	Notebook query	Observed (notebook)
Vector retrieval (production)	No record	API call	Not tested
Answer quality	Mean keyword accuracy 0.83 in the final run	Evaluation harness (local)	Evaluated (one run)
Frontend integration	One English and one Sinhala interaction	Screenshots	Observed (single)
Gateway forwarding	Path mapping matches; runtime untested	Source reading	Source-implemented
End-to-end farmer workflow	None conducted	User study	Not tested
Backend Jest suite	About 124 test blocks present; not executed here	`npm test`	Not executed
Load and performance in production	No record	None	Not tested

7.2 Unit and Evaluation Tests

The snapshot contains eight Jest files in `tests/` and three evaluation-related Jest files, about 124 `it` or `test` blocks by static count (90 and 34). They cover the chat route, chemical service, advisor handoff, taxonomy, embedder, follow-up consistency, language service, terminology resolver, query normalisation, scorer harmonisation and treatment safety. They were not executed for this report because dependencies and network access were unavailable, so no pass or fail result is reported.

The evaluation harness `tests/evaluation/evalRunner.js` calls the service function directly and was run four times on 6 and 7 October 2026. Table 12 gives the final run as reported in the thesis [2]. The thesis states its own limits: small samples, keyword-based scoring, scorers and ground truth that changed between runs, one run per configuration, and no record of the embedding provider used. Because the harness bypasses HTTP, the gateway and the frontend, it is not a deployment test.

Table 12. *Evaluation results of the final run (7 October 2026, 07:46), local harness*

Metric	Sample	Result	Threshold
Response accuracy (keyword score)	15 cases	Mean 0.83; 14 of 15 pass	0.75 per case
Retrieval relevance (top score)	10 cases	Mean 0.7531; 9 of 10 pass	0.70
Recommendation relevance	10 cases	8 of 10 pass	0.50 per case
Scope accuracy	15 cases	15 of 15 correct	0.90
Treatment safety flags	10 cases	0 flags	Zero expected
Mean response time	40 cases	10,162 ms (95th percentile 17,560 ms)	8,000 ms

7.3 Interpretation

The evidence supports the statement that the C4 service was built, started and reached its embedding provider on Railway on 8 October 2026 and that replies appeared in the public frontend in separate observations. It does not support a statement that the deployed chatbot has been verified end to end. The next verification step is a short scripted test of the public domain (`GET /health`, `GET /chat/topics`, one English and one Sinhala `POST /chat`) with the requests and responses saved.

8. Challenges and Resolutions

Table 13 summarises the engineering problems visible in the evidence, distinguishing those that were resolved in the code from those that remain open.

Table 13. *Challenges, impact and current approach*

Challenge	Impact	Resolution / current approach
Local embedding needs about 1-1.5 GB RAM (source comment)	A large model inside the Railway container	Query embedding moved to the Modal service; adds a network call, cold start and a second secret.
Dockerfile declares port 8004, log shows 8080	Apparent port mismatch	The service reads `PORT`; consistent with a platform-assigned port, but the variable is not visible (inference).
Sinhala questions with an English-only embedding model	Retrieval could fail on Sinhala text	Alias substitution plus LLM translation to English, falling back to the original query; adds latency and translation risk.
Unsafe or laboratory dosage figures in answers	Misleading treatment advice	Prompt rule 7 and extraction rules; the extraction prompt still allows "Approx." estimates, so the issue remains open and unvalidated by an expert.
Stored-vector pooling compatibility	Retrieval quality may be reduced	Open; verification script prepared but not run.
1,271 index records versus 1,250 recalculated chunks; index host differs from `.env.example`	Corpus-to-index correspondence unknown	Unresolved; no cause asserted.
Response time above target (mean 10.2 s against 8 s)	Poor interactivity	Open; up to three or four LLM calls per message; merging or parallelising calls is proposed.
Inconsistent component numbering (C3 in slides, C4 in service)	Confusion across documents	This report follows the gateway and service (C4).

Challenge	Impact	Resolution / current approach
Limits of the review evidence (repository not opened, build log tail only, masked values)	Some claims cannot be confirmed	Recorded as limitations and labelled in the text.

These issues shaped the final architecture. In particular, separating query embedding from the Railway container was a deployment decision, and it also isolates the embedding model so that a different pooling method or model can be tested while the Pinecone and LLM layers stay unchanged.

9. Current Status and Remaining Work

Table 14. *Current status of C4 work items*

Work item	Status	Notes
Express service and RAG pipeline	Implemented (source)	Routes, orchestration, retrieval and generation present in the snapshot.
Eight-topic taxonomy and scope gate	Implemented (source)	Keyword and alias match with LLM fallback; no rule engine.
English and Sinhala handling	Implemented (source)	One Sinhala interaction observed in the public frontend; correctness not assessed.
Modal embedding service	Deployed (observed)	Start-up warm-up completed; deployment record and dashboard not supplied.
Pinecone index	Exists (observed)	1,271 records; origin and pooling compatibility unresolved.
Railway deployment	Online / Active (observed)	Deployment `a1a12349`, 8 October 2026; trial allowance.
Frontend chat page and dosage calculator	Implemented (source)	Within the shared frontend; no user study.
Gateway forwarding to C4	Source-implemented	Runtime behaviour of the deployed gateway not tested.
Scripted tests of public endpoints and production retrieval	Pending	No requests or responses recorded.
Backend Jest suite	Present, not executed	About 124 test blocks by static count.
Expert review of dosage guidance	Pending	Sri Lankan registration status of active ingredients unchecked.
README, `.dockerignore`, request limits, authentication	Pending	Authentication and rate limiting belong at the gateway.
Hosting beyond the Railway trial	Pending	Costs for Railway, Modal and Pinecone not recorded.

9.1 Immediate Next Development Steps

1. Run and save a scripted check of the public domain (`GET /health`, `GET /chat/topics`, one English and one Sinhala `POST /chat`) and add it to the deployment record.
2. Run `npm test` in a connected environment, record the result and add an automated `/health` check to continuous integration.

3. Run the stored-vector verification script and, depending on the result, change the query pooling or re-ingest the corpus with a single documented method.
4. Record the deployed values of `EMBEDDING_PROVIDER`, `LLM_MODEL` and `PINECONE_*` in a non-secret deployment note and capture the Railway Settings tab (build, start and health-check configuration).
5. Decide whether to tighten the chemical-extraction prompt to remove model estimates, re-run the evaluation keeping both result sets, and obtain expert review of dosage guidance.
6. Add a README and a `.dockerignore`, move authentication and rate limiting to the gateway, and avoid logging message text.
7. Reduce latency by merging or parallelising LLM calls, measuring each stage; consider a shared or expiring session store if more than one instance is needed.
8. Plan hosting beyond the Railway trial and record the cost of each external service.

10. Research Evaluation Plan

The following evaluation is planned and has not been carried out. It is designed to remove the weaknesses of the existing harness, which used small samples, keyword scoring and scorers that changed between runs [2].

10.1 Test Discipline

Question sets, expected sources and scorers should be frozen before a comparison run, and every run should record the embedding provider, model, index record count, LLM identifier and date. Evaluation should be run against the deployed public domain as well as the service code, so that gateway, network and cold-start effects are included.

10.2 Retrieval and Answer Evaluation

Table 15. *Planned evaluation measures*

Area	Planned measure	Required output
Retrieval	Labelled English and Sinhala questions per topic with expected source documents; recall@k, precision@k and mean reciprocal rank; mean-pooled versus CLS-pooled query vectors after the verification script	Per-topic retrieval table and pooling decision
Answer faithfulness	Expert rubric checking that answers are supported by retrieved text, with keyword scores kept only as a secondary check	Rubric scores and reviewer notes
Dosage safety	Check extracted doses against product labels and Sri Lankan registration; count unsupported "Approx." values	List of flagged doses and reviewer decision
Scope and misuse	Out-of-scope and adversarial prompts (including prompt-injection cases listed by OWASP [16])	Refusal and leakage rates
Sinhala quality	Review of Sinhala replies by Sinhala-speaking reviewers, including terminology for chemicals and diseases	Reviewer ratings

10.3 Deployment and Usability Evaluation

Deployment evaluation should measure per-stage latency (normalisation, embedding, retrieval, each LLM call), the cold-start effect after the Modal scale-down of 300 s, and behaviour when Modal, Pinecone or OpenRouter is unavailable. A usability study with farmers or extension officers, if undertaken, should follow the approved ethics process, and proposal targets for response time and availability should remain targets until measured under a documented protocol.

10.4 Security and Operational Improvements

- Authenticate and rate-limit at the gateway, and limit request size and message length in C4.
- Set `ALLOWED_ORIGINS` explicitly and confirm it in the deployed configuration.
- Rotate any keys found in archived `.env` files, restrict key scopes, and avoid logging message text.
- Configure a health-check path and restart policy in Railway, and add retries for transient LLM errors.
- Use a shared store or expiry policy for sessions if more than one instance is required.

11. Conclusion

C4 has progressed from a notebook prototype to a Node.js retrieval-augmented advisory service that uses a Pinecone index for retrieval, a Modal-hosted BGE embedding service for queries and an OpenRouter-accessed language model for answers, with English and Sinhala handling and an in-memory conversation store. The source code, the Railway evidence and the thesis evaluation establish how the service is built and that it was built and started on Railway on 8 October 2026, with the Modal provider answering a start-up request.

The present evidence supports a conclusion of functional feasibility and successful deployment of a prototype. It does not establish that the deployed API, the production retrieval path or the end-to-end farmer workflow have been tested, and several issues remain open: the provenance and pooling of the stored vectors, dosage safety and expert validation, response time, documentation and hosting continuity. The next phase should therefore prioritise scripted tests of the public service, resolution of the vector-compatibility question and a more rigorous evaluation before final claims are made.

References

[1] B. K. Keshan, "PaddyGuard AI: A multimodal AI-based system for rice leaf disease and pest detection, treatment recommendation and knowledge assistance," Project Proposal Report R26-SE-015, Sri Lanka Inst. Inf. Technol., Mar. 2026.

[2] B. K. Keshan, "PaddyGuard AI: Treatment Advisory System and Agricultural Chatbot (C4), individual thesis," R26-SE-015, Sri Lanka Inst. Inf. Technol., Oct. 2026.

[3] Systems and Software Engineering - SQuaRE - System and Software Quality Models, ISO/IEC 25010:2011, 2011.

[4] OpenJS Foundation, "Node.js documentation." [Online]. Available: https://nodejs.org/docs [Accessed: date to be confirmed].

[5] StrongLoop and IBM, "Express - Node.js web application framework." [Online]. Available: https://expressjs.com [Accessed: date to be confirmed].

[6] OpenRouter, "OpenRouter documentation." [Online]. Available: https://openrouter.ai/docs [Accessed: date to be confirmed].

[7] Pinecone, "Pinecone documentation." [Online]. Available: https://docs.pinecone.io [Accessed: date to be confirmed].

[8] Modal Labs, "Modal documentation." [Online]. Available: https://modal.com/docs [Accessed: date to be confirmed].

[9] S. Xiao, Z. Liu, P. Zhang, and N. Muennighoff, "C-Pack: Packaged resources to advance general Chinese embedding," arXiv:2309.07597, 2023.

[10] Railway, "Railway documentation." [Online]. Available: https://docs.railway.com [Accessed: date to be confirmed].

[11] A. Wiggins, "The Twelve-Factor App," 2011. [Online]. Available: https://12factor.net [Accessed: date to be confirmed].

[12] A. Kumar et al., "Large scale validation and refinement of integrated pest management strategy for basmati rice (Oryza sativa) in farmers' participatory mode," Indian J. Agric. Sci., vol. 95, no. 12, pp. 1443-1448, Dec. 2025. (Knowledge-base file observed in Kaggle; bibliographic details as printed on its first page; DOI 10.56093/ijas.v95i12.163712 to be verified.)

[13] BAAI, "BGE explanation," FlagEmbedding tutorial. [Online]. Available: https://bge-model.com/tutorial/1_Embedding/1.2.3.html [Accessed: date to be confirmed].

[14] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), vol. 33, 2020, pp. 9459-9474.

[15] Docker Inc., "Dockerfile reference." [Online]. Available: https://docs.docker.com/reference/dockerfile/ [Accessed: date to be confirmed].

[16] OWASP Foundation, "OWASP Top 10 for Large Language Model Applications." [Online]. Available: https://owasp.org/www-project-top-10-for-large-language-model-applications/ [Accessed: date to be confirmed].

Appendix A - Current Component 4 API Contract

Table 16. *Chat request and reply fields*

Operation / field	Type or request	Meaning
`POST /chat` request	JSON	`message` (the farmer's text) and `session_id` (chosen by the client); both required, otherwise HTTP 400.
`reply`, `session_id`	string	Answer in the language of the question (or a fixed greeting or decline message), and an echo of the session identifier.
`language`	`en` or `si`	Language detected from the message characters.
`in_scope`	boolean	False for questions outside the eight topics.
`chemicals`	array	Chemical options extracted from the answer (name, dose per litre, coverage); empty if none or on extraction failure.
`follow_up_question`	string or null	One suggested follow-up question.
`sources`	array of strings	Source file name of each retrieved chunk (in-scope retrieval replies only).

Appendix B - Current Deployment Configuration (Non-secret)

Table 17. *Non-secret deployment identifiers*

Item	Value (source: Railway, Pinecone, source snapshot)
Railway project / environment	`handsome-renewal` / `production`
Service and domain	`paddyguard-treatment-advisory-chatbot`; `paddyguard-treatment-advisory-chatbot-production.up.railway.app`
Region and replicas	US East; 1 replica
Deployment identifier and time	`a1a12349`; 8 October 2026, approximately 01:20 (GMT+5:30)
Base image and start command	`node:20-slim`; `npm start` (`node server.js`)
Logged port	8080 (Dockerfile `EXPOSE 8004`)
Package	`treatment-advisory-chatbot` 2.0.0
Pinecone index	`rice-disease-advisor`, AWS us-east-1, 1,271 records, namespace `__default__`
Modal app (source)	`paddyguard-bge-embedder`

Table 18. *Configuration variables of the C4 service (names and public code defaults; deployed values masked)*

Variable	Code default	In Railway list	Secret
PORT	8004	No (list truncated)	No
ALLOWED_ORIGINS	*	Yes	No
OPENROUTER_API_KEY	none	Yes	Yes
OPENROUTER_BASE_URL	https://openrouter.ai/api/v1	Yes	No
LLM_MODEL	openai/gpt-4o-mini	Yes	No
PINECONE_API_KEY	none	Yes	Yes
PINECONE_HOST	none	Yes	No
PINECONE_INDEX_NAME	rice-disease-advisor	Yes	No
EMBEDDING_PROVIDER	local	Yes	No
EMBED_MODEL_NAME	Xenova/bge-large-en-v1.5	Yes	No
MODAL_EMBEDDING_URL	empty	Yes	No
MODAL_AUTH_TOKEN	empty	Yes	Yes
MODAL_TIMEOUT_MS	30000	Yes	No
RETRIEVAL_TOP_K	6	Yes	No
RETRIEVAL_TOP_K_BARE_TOPIC	8	Yes	No

Secret values such as API keys and bearer tokens are intentionally excluded from this report.

Appendix C - Development Evidence Summary

- Railway reported the service Online with an Active, successful deployment on 16 and 17 October 2026.
- Deployment `a1a12349` (8 October 2026) started the container with `npm start` and `node server.js`, logged port 8080, selected the Modal embedding provider and completed warm-up in 15.0 s.
- The build log tail shows an image push of 213.9 MB for the same deployment.
- The Pinecone dashboard showed index `rice-disease-advisor` with 1,271 records; a notebook check showed 35 unique source filenames.
- The final local evaluation run of 7 October 2026 is reported in the thesis; it is not a deployment test.
- One English (10 October) and one Sinhala (11 October) interaction were seen in the public frontend; correctness was not assessed.
- Not evidenced: public endpoint tests, production retrieval, end-to-end testing, expert validation of dosage guidance, and execution of the backend Jest suite.